GRADO: 2º Ingeniería del Software

ASIGNATURA: Análisis e ingeniería de requisitos

PRÁCTICA 1: APLICACIÓN SENDA URJC

Diego Ezquerra Barroso

Bárbara Cruz González

Sara El Moussaoui Houlbi

Javier Rodríguez Gil

Índice

1. Introducción 2
   1.1. Propósito 2
   1.2. Alcance 2
   1.3. Definiciones, siglas y abreviaturas 2
   1.4. Referencias 2
   1.5. Visión global 2
2. Descripción general 3
   2.1. Perspectiva del producto 3
   2.2. Funciones del producto 4
   2.3. Características del usuario 5
   2.4. Restricciones 6
   2.5. Supuestos y dependencias 6
3. Requisitos específicos 6
   3.1. Requisitos de la interfaz externa 6
      3.1.1. Interfaz con el usuario 6
      3.1.2. Interfaz con el hardware 7
      3.1.3. Interfaz con el software 7
      3.1.4. Interfaces de comunicaciones 7
   3.2. Requisitos funcionales 7
      3.2.1. Registro 7
      3.2.1.1 Requisito funcional 1: "Registro URJC" 7
      3.2.2. Inicio de sesión 8
   3.3 Requisitos de rendimiento 15
   3.4 Requisitos de diseño 15
   3.5 Atributos del sistema software 16

4. Anexo 16

1. Introducción

1.1. Propósito

Desarrollo de una aplicación para la Universidad Rey Juan Carlos que guíe a los usuarios a su destino priorizando la seguridad personal y la tranquilidad del usuario.

1.2. Alcance

El nombre del producto es Senda URJC. Está aplicación está destinada al ámbito social y de seguridad ciudadana, donde los usuarios tendrán la opción de ver la ruta más segura para llegar de un punto origen a un punto destino. A través de esta aplicación, también podrán solicitar un acompañante para el camino y estar al tanto de las zonas a evitar de los campus de la URJC. El objetivo principal de esta aplicación es reforzar la seguridad de los alumnos durante los desplazamientos.

1.3. Definiciones, siglas y abreviaturas

- **PAS:** Personal de Administración y Servicios
- **PDI:** Personal Docente e Investigador
- **URJC:** Universidad Rey Juan Carlos
- **LOPD:** Ley Orgánica de Protección de Datos
- **PERSEIDAS** (Percepción de Seguridad del Estudiantado en sus Itinerarios de Desplazamiento a las Aulas)
- **PTGAS:** Personal Técnico, de Gestión y de Administración y Servicios
- **GPS:** Sistema de Posicionamiento Global en inglés Global Positioning System
- **SSO:** Sistema de Autenticación Centralizada

1.4. Referencias

Las referencias que hemos usado para la especificación de requisitos de esta aplicación han sido: el material disponible en el aula virtual como la guía IEEE-29148-2018, el tema 3, el tema 4, el tema 5, los ejercicios realizados en clase…

1.5. Visión global

A partir de este punto, el documento consta de dos partes. En la primera parte, se muestra una descripción general donde se desarrolla la perspectiva y funciones del producto, además de las características del usuario, las restricciones, supuestos y dependencias de Senda URJC.

En la segunda parte se describen los requisitos funcionales, no funcionales y se muestra el anexo con los diagramas de casos de uso y la tabla extendida de cada uno de estos.

2. Descripción general

2.1. Perspectiva del producto

Senda URJC se presenta como solución de movilidad integrada en la comunidad universitaria. La aplicación formará parte del proyecto de investigación PERSEIDAS. Al pertenecer a este proyecto y depender del servidor de la Universidad no es una aplicación independiente. Se necesita el servidor de Lumen Smart S.A. para poder acceder a los datos del estado de las farolas y el de la universidad para los usuarios. Además la aplicación debe estar operativa mínimo durante el horario de las universidades cuando aún puede quedar gente en los campus.

Interfaces del sistema: la aplicación presenta varias pantallas a los usuarios, como: la pestaña "Inicio" donde el usuario podrá ver el mapa del campus en el que se encuentre junto con las distintas opciones para iniciar una ruta. Por otro lado la pestaña para registro de los usuarios de la universidad donde se pedirán los datos personales al usuario vinculado a la URJC. También la pestaña de voluntarios donde se gestionan acompañamientos. Además de la pantalla de notificaciones donde se pueden reportar cambios en el estado de los caminos o del entorno. Finalmente la pestaña de login donde el usuario podrá acceder a la aplicación introduciendo su dirección de correo y su contraseña.

Interfaces de usuario: Nuestra aplicación está dirigida a dispositivos móviles ya sean android o ios. Esta tendrá un diseño visual acorde al manual de identidad visual corporativo de la URJC. Además de cumplir con el manual se usará un modelo inclusivo de colores y funcionalidades fáciles de usar en momentos de estrés donde no se puede mirar directamente la pantalla ni usar las dos manos. Por otro lado, la aplicación contará con dos opciones de idioma a elegir entre español e inglés.

Interfaces Software: La aplicación Senda URJC dependerá del servidor de la Universidad Rey Juan Carlos para llevar a cabo la autenticación de doble factor por el SSO de la universidad. Por otra parte, la aplicación se conecta con el servidor de Lumen Smart S.A. para recibir los datos del estado lumínico de las farolas, afluencia y el estado del entorno. Para almacenar los datos de los usuarios externos la aplicación se conecta con una base de datos de la universidad. Para las funcionalidades de geolocalización se recurrirá al medio favorito del usuario según el sistema operativo. Por último para la gestión de incidencias se usará una base de datos comunicada con los servidores de Lumen Smart S.A.

Interfaces Hardware: La aplicación precisará de un receptor GPS para tener acceso a la ubicación precisa de los usuarios durante los trayectos y para localizarlos en el campus. También se necesita un Actuador Háptico para controlar la vibración del dispositivo en el momento que se necesite además de un sensor de iluminación para usar la linterna del móvil en las incidencias.

Interfaces de Comunicaciones:Senda URJC precisará de conexión a internet 4G, 5G o wifi para el envío de la ubicación, enviar y recibir datos de Lumen Smart S.A. entre otras funcionalidades. También se exige compatibilidad con servicios de mensajería instantánea para el envío de ubicaciones a contactos de seguridad o notificar de incidencias. Por último se pide que haya seguridad y privacidad en toda transferencia de datos sensibles que trate la aplicación

2.2. Funciones del producto

Las funciones básicas de la aplicación SendaURJC son el establecimiento de rutas seguras e inicio de sesión de los usuarios, la posibilidad de dichos usuarios de poder pedir o ser un acompañamiento.

Para que los usuarios puedan registrarse, la aplicación les permite introducir sus datos. Para usuarios los datos pedidos serán: correo electrónico y contraseña de la institución o bien acceder por el SSO de la universidad.

A continuación, la aplicación comprobará que el correo electrónico es válido y si el usuario está registrado. En ese caso de que no lo esté se creará el perfil del nuevo usuario de la aplicación. Si si lo está se accedera a su perfil.

En el caso de que el correo o la contraseña no sean válidos, la aplicación mostrará una notificación informando al usuario acerca del error.

La aplicación permitirá el inicio de sesión para usuarios que tengan correo de alumnos, profesor o empleado de la universidad. Para ello los usuarios pertenecientes a la URJC deberán de introducir su usuario y contraseña, además por seguridad se va a implementar un doble factor de autenticación mediante el sistema externo de Microsoft Authenticator.

Al entrar a la aplicación el usuario verá un mapa de la zona en la que se encuentra. En esta pantalla podrá elegir un punto de inicio de ruta y un punto de finalización de la ruta. En el caso de que la ruta se quiera empezar en otro momento se podrá elegir la hora de salida y dejar la ruta programada. También se cuenta con un botón para seleccionar si se quiere ir acompañado de un voluntario. En caso de marcar esta opción se mandará una solicitud de acompañamiento a los voluntarios. Una vez se hayan seleccionado estas opciones se le podrá dar al botón de iniciar ruta en la pantalla inicial. Una vez se presione la aplicación ofrece distintas rutas con una valoración y un color asociado marcando qué rutas son más o menos seguras. Una vez el usuario elija qué ruta quiere seguir tendrá que seleccionarla y presionar continuar. La aplicación redirigirá a la pantalla de ruta donde va recibiendo las indicaciones para llegar al destino. En esta pantalla el usuario tendrá los botones de peligro en la disposición que haya indicado. También se contará con un pequeño menú desplegable para indicar incidencias de luz afluencia u obstáculos. Al final del trayecto saldrá una notificación preguntando si se llegó bien al destino.

Por otro lado se contará con otra pantalla accesible desde el inicio para los usuarios que se ofrezcan como voluntarios para acompañar durante rutas. En esta pantalla podrá ver solicitudes de otros usuarios y aceptarlas o rechazarlas. Una vez aceptadas se dará información extra a las dos partes. En caso de ser rechazada la solicitud no aparecerá más en la aplicación del voluntario.

Por último en la pantalla de inicio habrá un botón para ver el perfil del usuario. En esta pantalla se podrán ver las estadísticas del usuario además de los datos ya introducidos y faltantes. Habrá un botón para editar los datos.

2.3. Características del usuario

Se pueden identificar varios grupos de distintos perfiles de usuarios para nuestra aplicación. Entre ellos:

2.3.1 Comunidad Universitaria (Usuarios Finales)

Este grupo es el más amplio y para el que está principalmente pensada la aplicación. Podemos encontrar a los estudiantes que usaron la aplicación para traslados entre aularios o salida del campus. Las horas críticas de este perfil sería principalmente entre clases y mayoritariamente a la salida del turno de tarde. Por otro lado se incluyen los PDI los cuales tendrán un horario crítico parecido al de los estudiantes. Por último están los PTGAS que podrían ir más orientados al cierre del campus.

2.3.2 Personas Voluntarias

Este perfil aunque también son estudiantes,PDI o PTGAS tienen que tener un cierto compromiso con los demás usuarios de la aplicación y cumplir en la medida de los posible con acompañamientos.

Deberán conocer lo mejor posible los campus sobre todo los que frecuenten y más acompañamientos vayan a hacer. También sería muy positivo que mínimo hablasen inglés y español para poder comunicarse con los usuarios.
Usaron la aplicación para comunicarse con los usuarios y deberían tener su información completa.

2.3.3 Administradores del Sistema

Este perfil serían los administradores de la aplicación que la mantienen, solucionan errores, gestionan tickets y aseguran el correcto funcionamiento de la aplicación. Deberán saber gestionar las bases de datos, controlar las conexiones con los distintos servidores y programar para solucionar y mejorar funciones.

2.3.4 Personal de Seguridad de la URJC

No son como tal usuarios de la aplicación pero cumplen un papel fundamental para responder a incidencias y mantener y promover la seguridad en los campus.

2.4. Restricciones

La aplicación Senda URJC solo funcionará en los campus de la universidad y alrededores para puntos de interés ya sean estaciones o paradas.

Otro punto es que la aplicación solo funciona en dispositivos móviles con sistema operativo Android o iOS. Además se tienen que cumplir con la protección de datos y cumplir con el manual de identidad visual corporativo de la URJC más reciente.

2.5. Supuestos y dependencias

Es necesario que los usuarios tengan conexión a internet. Por último se necesita que los usuarios tengan un servicio de geolocalización activo y preciso.

Por otro lado, se necesita el correcto funcionamiento de las conexiones entre servidores y el acceso a datos de las bases de datos. Además del funcionamiento de los sensores repartidos por los campus.

3. Requisitos específicos

3.1. Requisitos de la interfaz externa

3.1.1. Interfaz con el usuario

Existe una relación entre la aplicación Senda URJC y el usuario cuando este solicita una ruta, consulta las rutas seguras disponibles, pide acompañamiento, reporta incidencias del entorno, activa el modo "Voy contigo" o recibe una prealerta por detención inesperada o desviación de la ruta. El enunciado establece expresamente que la aplicación deberá calcular rutas seguras, permitir solicitar nuevas rutas antes o durante el trayecto, permitir reportar incidencias y emitir una prealerta si el usuario se detiene inesperadamente o se desvía bruscamente de la ruta.

3.1.2. Interfaz con el hardware

La aplicación Senda URJC no interactúa directamente con el hardware de las luminarias ni de los sensores. La captura de datos del alumbrado inteligente y de los sensores de presencia será responsabilidad del sistema externo LumenSmart, que enviará la información procesada a Senda URJC.

3.1.3. Interfaz con el software

Existe una interfaz software entre Senda URJC y varios sistemas externos:

- El sistema de autenticación centralizada de la URJC (SSO), ya que el enunciado exige autenticación con credenciales corporativas;
- El sistema LumenSmart, para recibir datos de iluminación, afluencia, estado operativo y alertas críticas;
- Y los componentes internos del sistema que gestionen rutas, incidencias, acompañamientos y administración.

3.1.4. Interfaces de comunicaciones

Existen interfaces de comunicaciones entre Senda URJC y sistemas externos para:

- Autenticación de usuarios,
- Obtención de datos del alumbrado inteligente,
- Recepción de alertas en tiempo real,
- Y gestión de incidencias y acompañamiento.

El enunciado establece que LumenSmart enviará a Senda URJC información en tiempo real sobre el estado de las farolas y que, si un tramo sufre un apagón, Senda URJC deberá recibir la alerta automáticamente.

3.2. Requisitos funcionales

3.2.1. Recepción de datos de luminarias

3.2.1.1. Requisito funcional 1: "Recepción del estado operativo de luminarias"

- **Descripción**: El sistema deberá recibir y registrar el estado operativo de cada luminaria proporcionado por el sistema LumenSmart.
- **Entradas**: Identificador único de luminaria, y Estado operativo ∈ {ENCENDIDA, APAGADA, AVERIADA, EN_MANTENIMIENTO}.
- **Salida**: Estado operativo de la luminaria almacenado y actualizado en el sistema.
- **Posibles errores**: Identificador de luminaria inexistente, Estado operativo no reconocido, y Mensaje recibido con formato inválido.
- **Método de verificación:** Prueba funcional mediante recepción de mensajes desde LumenSmart.

3.2.1.2. Requisito funcional 2: "Recepción de la intensidad lumínica"

- **Descripción**: El sistema deberá recibir y almacenar la intensidad lumínica de cada luminaria expresada como porcentaje.
- **Entradas**: Identificador único de luminaria, e Intensidad lumínica (valor numérico entre 0 y 100).
- **Salida**: Intensidad lumínica registrada y asociada a la luminaria correspondiente.
- **Posibles errores**: Valor fuera de rango, Identificador inexistente, y Formato de dato inválido.
- **Método de verificación:** Prueba funcional mediante recepción de mensajes desde LumenSmart.

3.2.1.3. Requisito funcional 3: "*Recepción de iluminación ambiental*"

- **Descripción**: El sistema deberá recibir y registrar los valores de iluminación ambiental medidos en lux para cada luminaria.
- **Entradas**: Identificador único de luminaria, y Valor de iluminancia ambiental (lux).
- **Salidas**: Nivel de iluminación ambiental registrada en el sistema.
- **Posibles errores**: Valor no numérico, Identificador inexistente, y Unidad de medida incorrecta.
- **Método de verificación:** Prueba de integración con sensores de LumenSmart.

3.2.1.4. Requisito funcional 4: "Recepción de eventos de detección de presencia"

- **Descripción:** El sistema deberá recibir y registrar eventos binarios de detección de presencia procedentes de los sensores gestionados por LumenSmart.
- **Entradas:** Identificador de sensor o luminaria, y Evento de presencia (PRESENCIA / NO_PRESENCIA).
- **Salida:** Evento de presencia registrado en el sistema.
- **Posibles errores:** Evento no reconocido, Identificador inexistente, y Evento duplicado.
- **Método de verificación:** Prueba funcional con eventos simulados.

3.2.1.5. Requisito funcional 5: "Recepción de activaciones agregadas de sensores"

- **Descripción:** El sistema deberá recibir y almacenar contadores agregados de activaciones de sensores de presencia en intervalos temporales definidos.
- **Entradas:** Identificador de sensor o zona, Número de activaciones, e Intervalo temporal asociado.
- **Salida:** Contador de activaciones almacenado para el intervalo indicado.
- **Posibles errores:** Intervalo temporal inválido, Contador negativo, e Identificador inexistente.
- **Método de verificación:** Prueba de almacenamiento y consistencia temporal.

3.2.1.6. Requisito funcional 6: "Recepción del índice de afluencia por zona"

- **Descripción:** El sistema deberá recibir y registrar un índice de afluencia agregado por zona del campus, normalizado en un rango de 0 a 100.
- **Entradas:** Identificador único de zona, e Índice de afluencia $\in$ [0,100].
- **Salida**: Índice de afluencia asociado a la zona correspondiente.
- **Posibles errores**: Zona inexistente, Valor fuera de rango, y Dato ausente o incompleto.
- **Método de verificación:** Prueba de validación de rangos y asociación correcta.

3.2.1.7. Requisito funcional 7: "Identificación única de luminarias"

- **Descripción**: El sistema deberá identificar cada luminaria mediante un identificador alfanumérico único y persistente proporcionado por LumenSmart.
- **Entradas**: Identificador alfanumérico de luminaria.
- **Salida**: Luminaria registrada o sincronizada con su identificador único.
- **Posibles errores**: Identificador duplicado, Identificador vacío, y Formato inválido.
- **Método de verificación:** Inspección de unicidad y persistencia del identificador.

3.2.1.8. Requisito funcional 8: "Identificación de zonas del campus"

- **Descripción**: El sistema deberá identificar y registrar cada zona del campus mediante un identificador único proporcionado por LumenSmart.
- **Entradas**: Identificador único de zona.
- **Salida**: Zona registrada y disponible para asociación con luminarias.
- **Posibles errores**: Identificador duplicado, Identificador inexistente, y Formato inválido.
- **Método de verificación:** Prueba de registro y consulta de zonas.

3.2.1.9. Requisito funcional 9: "Asociación de luminarias a zonas"

- **Descripción**: El sistema deberá asociar cada luminaria registrada a una zona del campus definida por LumenSmart.
- **Entradas**: Identificador de luminaria, e Identificador de zona.
- **Salida**: Asociación luminaria-zona almacenada correctamente.
- **Posibles errores:** Luminaria inexistente, Zona inexistente, y Asociación duplicada o ambigua.
- **Método de verificación:** Prueba de consistencia de asociaciones.

3.2.2. Integración con LumenSmart

3.2.2.1. Requisito funcional 10: "Recepción de alertas críticas de alumbrado"

- **Descripción:** El sistema deberá recibir y registrar alertas críticas relacionadas con apagones, averías o cambios críticos de estado en luminarias o zonas.
- **Entradas:** Tipo de alerta, Identificador de luminaria o zona, y Marca temporal.
- **Salida:** Alerta crítica registrada y disponible para su tratamiento.
- **Posibles errores:** Tipo de alerta desconocido, Identificador inexistente, y Alerta duplicada.
- **Método de verificación:** Prueba de recepción y registro de alertas.

3.2.2.2. Requisito funcional 11: "Consulta de datos mediante API REST"

- **Descripción:** El sistema deberá consultar periódicamente los sistemas de LumenSmart mediante una API REST con el fin de obtener información de estados, listados e históricos.
- **Entradas:** Solicitud de consulta, Endpoint REST, Parámetros de consulta, y Credenciales válidas.
- **Salida**: Respuesta de la API con los datos solicitados.
- **Posibles errores:** API no disponible, Tiempo de espera agotado, Autenticación fallida, Respuesta inválida o incompleta.
- **Método de verificación:** Prueba de integración mediante llamadas REST simuladas y reales.

3.2.2.3. Requisito funcional 12: "*Procesamiento del estado de mantenimiento*"

- **Descripción**: El sistema deberá procesar el estado EN_MANTENIMIENTO de las luminarias recibido desde LumenSmart y reflejarlo en su estado interno.
- **Entradas**: Identificador único de luminaria, y Estado de mantenimiento.
- **Salida**: Luminaria marcada como *en mantenimiento* en el sistema.
- **Posibles errores**: Identificador inexistente, Valor de estado inválido, e Incoherencia con el estado actual.
- **Método de verificación:** Prueba funcional de transición de estados.

3.2.2.4. Requisito funcional 13: "*Recepción de datos energéticos*"

- **Descripción:** El sistema deberá recibir y registrar los datos energéticos de cada luminaria, incluyendo potencia instantánea y consumo acumulado.
- **Entradas:** Identificador único de luminaria, Potencia instantánea, y Consumo acumulado.
- **Salida:** Datos energéticos registrados y asociados a la luminaria correspondiente.
- **Posibles errores:** Valores negativos, Identificador inexistente, y Unidad de medida incorrecta.
- **Método de verificación:** Prueba de validación de datos y persistencia.

3.2.2.5. Requisito funcional 14: *"Intercambio de información en formato JSON"*

- **Descripción:** El sistema deberá intercambiar información con LumenSmart utilizando mensajes en formato JSON codificados en UTF-8.
- **Entradas:** Mensajes de entrada y salida del intercambio de datos.
- **Salida:** Mensajes procesados o generados en formato JSON UTF-8 válido.
- **Posibles errores:** JSON malformado, Codificación incorrecta, y Campos obligatorios ausentes.
- **Método de verificación:** Inspección sintáctica y prueba de compatibilidad de mensajes.

3.2.2.6. Requisito funcional 15: *"Recepción de eventos mediante webhooks"*

- **Descripción:** El sistema deberá recibir eventos en tiempo real procedentes de LumenSmart mediante notificaciones webhook.
- **Entradas:** Notificación webhook, Cabeceras HTTP, y Cuerpo del evento.
- **Salida:** Evento recibido y disponible para su procesamiento.
- **Posibles errores:** Endpoint inaccesible, Mensaje duplicado, y Formato inválido.
- **Método de verificación:** Prueba funcional con webhooks simulados.

3.2.3. Seguridad de la integración

3.2.3.1. Requisito funcional 16: "*Autenticación con OAuth 2.0*"

- **Descripción:** El sistema deberá autenticarse frente a las API de LumenSmart utilizando el flujo OAuth 2.0 Client Credentials.
- **Entradas:** Client ID, Client Secret, y Endpoint de autenticación.
- **Salida:** Token de acceso válido para consumir la API.
- **Posibles errores:** Credenciales inválidas, Token expirado, y Servidor de autenticación no disponible.
- **Método de verificación:** Prueba de autenticación y renovación de token.

3.2.3.2. Requisito funcional 17: "*Validación de integridad y autenticidad de eventos*"

- **Descripción:** El sistema deberá validar la integridad y autenticidad de los eventos webhook recibidos mediante firmas HMAC-SHA256.
- **Entradas:** Evento recibido, Firma asociada, y Clave compartida de validación.
- **Salida:** Evento validado o rechazado por el sistema.
- **Posibles errores:** Firma ausente, Firma inválida, y Discrepancia entre el contenido y la firma.
- **Método de verificación:** Prueba criptográfica de validación de firmas.

3.2.3.3. Requisito funcional 18: "*Procesamiento de eventos críticos en tiempo real*"

- **Descripción:** El sistema deberá procesar los eventos críticos de alumbrado con una latencia de extremo a extremo inferior a 10 segundos desde su recepción.
- **Entradas:** Evento crítico, Marca temporal de detección, y Datos asociados al evento.
- **Salida:** Evento crítico procesado y disponible en el sistema dentro del tiempo establecido.
- **Posibles errores:** Latencia superior al umbral permitido, Sobrecarga del sistema, y Marca temporal ausente o inválida.
- **Método de verificación:** Prueba de rendimiento y medición de latencia.

3.2.4. Tratamiento y almacenamiento de datos

3.2.4.1. Requisito funcional 19: "*Procesamiento de datos según frecuencia de actualización*"

- **Descripción:** El sistema deberá procesar los datos entrantes de acuerdo con frecuencias de actualización predefinidas según el tipo de dato recibido.
- **Entradas:** Tipo de dato, Marca temporal, y Dato recibido.
- **Salida:** Dato procesado conforme a la frecuencia establecida.
- **Posibles errores:** Marca temporal ausente, Dato recibido fuera de la frecuencia esperada, y Tipo de dato no reconocido.

- **Método de verificación:** Prueba de control temporal y reglas de procesamiento.

3.2.4.2. Requisito funcional 20: "*Almacenamiento de datos históricos*"

- **Descripción:** El sistema deberá almacenar de forma persistente los datos históricos validados procedentes de los sensores y zonas gestionados por LumenSmart.
- **Entradas**: Datos validados, y Marca temporal.
- **Salida**: Registro histórico almacenado en el sistema.
- **Posibles errores**: Fallo de almacenamiento, Registro duplicado, y Conflicto con la política de retención de datos.
- **Método de verificación:** Prueba de persistencia y comprobación de retención.

3.2.4.3. Requisito funcional 21: "*Consulta interna de datos*"

- **Descripción:** El sistema deberá permitir la consulta de datos actuales e históricos de iluminación y afluencia por parte de los componentes internos autorizados del sistema.
- **Entradas:** Solicitud interna de consulta, y Filtros temporales y/o espaciales.
- **Salida:** Conjunto de datos actual o histórico solicitado.
- **Posibles errores:** No existen datos para la consulta, Filtros inválidos, y Componente solicitante no autorizado.
- **Método de verificación:** Prueba funcional mediante consultas internas simuladas.

3.2.4.4. Requisito funcional 22: "*Marcado de tramos con información parcial*"

- **Descripción:** El sistema deberá clasificar los tramos del campus como de información parcial cuando la cobertura de sensores disponible sea insuficiente.
- **Entradas:** Datos de cobertura de sensores, e Identificador de tramo o zona.
- **Salida:** Tramo o zona marcado como de información parcial o baja confianza.
- **Posibles errores:** Tramo inexistente, Ausencia de metadatos de cobertura, e Inconsistencia en la zonificación.
- **Método de verificación:** Inspección de clasificación de tramos ante pérdida de cobertura.

3.2.4.5. Requisito funcional 23: "*Clasificación de la calidad de los datos*"

- **Descripción:** El sistema deberá clasificar el nivel de confianza de los datos recibidos y deberá utilizar dicha clasificación para influir en el cálculo y selección de rutas.
- **Entradas:** Datos de sensores, Metadatos de cobertura, y Reglas de calidad o confianza.
- **Salida:** Nivel de confianza asignado, y Nivel de confianza aplicado al cálculo de rutas.
- **Posibles errores:** Falta de metadatos, Criterios de calidad no disponibles, y Datos inconsistentes.

- **Método de verificación:** Prueba funcional del impacto del nivel de confianza en el enrutado.

3.2.5. Uso de datos en el cálculo de rutas

3.2.5.1. Requisito funcional 24: "*Recalcular rutas dinámicamente*"

- **Descripción:** El sistema deberá recalcular de forma dinámica una ruta activa cuando se reciban nuevos datos de iluminación o afluencia que afecten a su nivel de seguridad.
- **Entradas:** Ruta activa, y Nuevos datos de iluminación y afluencia.
- **Salida:** Ruta recalculada, y Actualización de la ruta actual.
- **Posibles errores:** No existe ruta alternativa, Datos insuficientes para recalcular, y Ruta activa inexistente.
- **Método de verificación:** Prueba funcional con cambio dinámico de condiciones.

3.2.6. Servicios

3.2.6.1. Requisito funcional 25: "*Guiado del usuario hasta su destino*"

- **Descripción:** La aplicación deberá guiar al usuario desde un punto de origen hasta un punto de destino a través de una ruta seleccionada.
- **Entradas:** Punto origen, Punto destino, y Condiciones de cálculo de ruta.
- **Salida:** Instrucciones de guiado durante el trayecto.
- **Posibles errores:** Punto de destino incorrecto o inaccesible.
- **Método de verificación:** Prueba funcional de guiado completo.

3.2.6.2. Requisito funcional 26: "*Obtención de la posición del usuario*"

- **Descripción:** La aplicación deberá obtener la posición del usuario mediante servicios de geolocalización del dispositivo.
- **Entradas:** Señal de geolocalización del dispositivo.
- **Salida:** Posición actual del usuario representada en el mapa.
- **Posibles errores:** Señal de geolocalización insuficiente o imprecisa.
- **Método de verificación:** Prueba funcional de localización en distintos entornos.

3.2.6.3. Requisito funcional 27: "*Acceso mediante autenticación centralizada*"

- **Descripción:** La aplicación deberá permitir el acceso únicamente a usuarios autenticados mediante el sistema de autenticación centralizada de la URJC.
- **Entradas:** Credenciales institucionales del usuario.
- **Salida:** Acceso concedido a la aplicación.
- **Posibles errores:** Credenciales inválidas, y Error en el sistema de autenticación.
- **Método de verificación:** Prueba funcional de acceso autorizado y no autorizado.

3.2.6.4. Requisito funcional 28: "*Perfil único*"

- **Descripción:** La aplicación deberá mantener un perfil único asociado a cada usuario autenticado.
- **Entradas:** Datos de perfil de usuario.
- **Salida:** Perfil de usuario almacenado y accesible.
- **Posibles errores:** Intento de duplicación de perfiles, y Solapamiento de datos de usuario.
- **Método de verificación:** Inspección de unicidad e integridad del perfil.

3.2.6.5. Requisito funcional 29: "*Selección de origen y destino*"

- **Descripción:** La aplicación deberá permitir al usuario seleccionar un punto de origen y un punto de destino para el cálculo de una ruta.
- **Entradas:** Punto de origen seleccionado, y Punto de destino seleccionado.
- **Salida:** Solicitud válida de cálculo de rutas.
- **Posibles errores:** Selección errónea de origen o destino.
- **Método de verificación:** Prueba funcional de selección y validación de puntos.

3.2.6.6. Requisito funcional 30: "*Mostrar múltiples rutas al usuario*"

- **Descripción:** La aplicación deberá calcular varias rutas y mostrarlas al usuario.
- **Entradas:** Punto de origen, Punto de destino, y Datos actuales de sensores de iluminación, afluencia y estado del entorno.
- **Salida:** Conjunto de rutas alternativas mostradas al usuario.
- **Posibles errores:** Imposibilidad de calcular las rutas válidas, y Datos de sensores insuficientes o no disponibles.
- **Método de verificación:** Prueba funcional de generación y visualización de rutas.

3.2.6.7. Requisito funcional 31: "*Cálculo del índice de seguridad de una ruta*"

- **Descripción:** La aplicación deberá calcular un índice de seguridad asociado a cada ruta generada.
- **Entradas:** Datos de sensores asociados a la ruta.
- **Salida:** Índice peligrosidad calculado para cada ruta.
- **Posibles errores:** Datos insuficientes para el cálculo, y Uso de datos no actualizados.
- **Método de verificación:** Prueba funcional de cálculo del índice peligrosidad.

3.2.6.8. Requisito funcional 32: "*Composición del índice de seguridad*"

- **Descripción:** El índice de seguridad deberá calcularse a partir del nivel de iluminación, la afluencia de personas y el estado del entorno de cada tramo de la ruta.
- **Entradas:** Datos de iluminación, Datos de afluencia, y Datos del estado del entorno.
- **Salida:** Índice de seguridad resultante.
- **Posibles errores:** Datos de sensores incompletos, y Datos no actualizados.
- **Método de verificación:** Inspección de las variables utilizadas en el cálculo del índice.

3.2.6.9. Requisito funcional 33: "*Prioridad de calles con alumbrado activo*"

- **Descripción:** El índice deberá priorizar los tramos de ruta que dispongan de alumbrado activo en el cálculo del índice de seguridad.
- **Entradas:** Datos del estado del alumbrado.
- **Salida:** Valor de prioridad aplicado a los tramos iluminados.
- **Posibles errores:** Información de alumbrado no disponible o desactualizada.
- **Método de verificación:** Prioridad de calles con alumbrado activo.

3.2.6.10. Requisito funcional 34: "*Prioridad de zonas con mayor afluencia*"

- **Descripción:** El sistema deberá priorizar los tramos de ruta con mayor afluencia de personas en el cálculo del índice de seguridad.
- **Entradas:** Datos de afluencia de sensores.
- **Salida:** Valor de prioridad aplicado a los tramos con mayor afluencia.
- **Posibles errores:** Datos de afluencia no disponibles o desactualizados.
- **Método de verificación:** Prueba funcional con distintos niveles de afluencia.

3.2.6.11. Requisito funcional 35: "*Penalización de zonas con obstáculos*"

- **Descripción:** El sistema deberá penalizar los tramos de ruta que presenten obras, obstáculos o incidencias reportadas en el entorno.
- **Entradas:** Datos de sensores, e Incidencias registradas.
- **Salida:** Penalización aplicada al índice de seguridad del tramo.
- **Posibles errores:** Datos no actualizados, e Incidencias no registradas correctamente.
- **Método de verificación:** Prueba funcional con incidencias simuladas.

3.2.6.12. Requisito funcional 36: "*Identificación de la ruta más segura*"

- **Descripción:** La aplicación deberá identificar la ruta con mayor índice de seguridad entre todas las rutas calculadas.
- **Entradas:** Conjunto de rutas con sus índices de seguridad.
- **Salida:** Ruta identificada como la más segura.
- **Posibles errores:** Datos de sensores no actualizados, e Índices no disponibles.

- **Método de verificación:** Prueba funcional de comparación de índices.

3.2.6.13. Requisito funcional 37: "*Oferta de rutas alternativas*"

- **Descripción:** La aplicación deberá mostrar rutas alternativas adicionales, ordenadas según su índice de seguridad.
- **Entradas:** Conjunto de rutas calculadas.
- **Salida:** Lista ordenada de rutas alternativas.
- **Posibles errores:** Rutas alternativas no disponibles, e Información incompleta de seguridad.
- **Método de verificación:** Prueba funcional de ordenación y presentación de rutas.

3.2.6.14. Requisito funcional 38: "*Selección de ruta por el usuario*"

- **Descripción:** La aplicación deberá permitir al usuario seleccionar una de las rutas ofrecidas para iniciar el trayecto.
- **Entradas:** Ruta seleccionada por el usuario.
- **Salida:** Ruta seleccionada establecida como ruta activa.
- **Posibles errores:** Selección de una ruta no válida o inexistente.
- **Método de verificación:** Prueba funcional de selección y activación de ruta.

3.2.6.15. Requisito funcional 39: "*Compartir la ubicación en tiempo real*"

- **Descripción:** La aplicación deberá permitir al usuario compartir su ubicación en tiempo real con un destinatario autorizado.
- **Entradas:** Ubicación actual del usuario, y Destinatario seleccionado.
- **Salida:** Confirmación de que la ubicación está siendo compartida.
- **Posibles errores:** Fallo en el envío de la ubicación, y Destinatario no accesible.
- **Método de verificación:** Prueba funcional de compartición de ubicación en tiempo real.

3.2.6.16. Requisito funcional 40: "*Configuración de contacto de confianza*"

- **Descripción:** La aplicación deberá permitir al usuario configurar un contacto de confianza asociado a su perfil.
- **Entradas:** Datos del contacto de confianza seleccionado.
- **Salida:** Confirmación de que el contacto de confianza ha sido registrado correctamente.
- **Posibles errores:** Fallo a la hora de guardar el contacto de confianza, y Datos del contacto incompletos o inválidos.
- **Método de verificación:** Prueba funcional de creación y persistencia del contacto de confianza.

3.2.6.17. Requisito funcional 41: "*Cambio de ruta antes de iniciar el trayecto*"

- **Descripción:** La aplicación deberá permitir al usuario cambiar la ruta seleccionada antes de iniciar el trayecto.
- **Entradas:** Solicitud de cambio de ruta por parte del usuario.
- **Salida:** Nueva ruta seleccionada establecida como ruta activa.
- **Posibles errores:** Fallo a la hora de cancelar la ruta previamente seleccionada, y No disponibilidad de rutas alternativas.
- **Método de verificación:** Prueba funcional de sustitución de ruta previa al inicio.

3.2.6.18. Requisito funcional 42: "*Reporte de incidencias por el usuario*"

- **Descripción:** La aplicación deberá permitir al usuario reportar incidencias que afecten a la percepción de seguridad del entorno.
- **Entradas:** Datos de la incidencia reportada.
- **Salida:** Incidencia registrada en el sistema.
- **Posibles errores:** Fallo en el registro de la incidencia, y Datos de incidencia incompletos.
- **Método de verificación:** Prueba funcional de creación y almacenamiento de incidencias.

3.2.6.19. Requisito funcional 43: "*Panel de administración para gestión de incidencias*"

- **Descripción:** El sistema deberá proporcionar un panel de administración para la gestión y resolución de incidencias reportadas.
- **Entradas:** Incidencias y notificaciones registradas.
- **Salida:** Estado actualizado de las incidencias gestionadas.
- **Posibles errores:** Error al acceder al panel de administración, y Fallo en la gestión de incidencias.
- **Método de verificación:** Prueba funcional del panel de administración.

3.2.6.20. Requisito funcional 44: "*Generación automática de tickets*"

- **Descripción:** El sistema deberá generar automáticamente un ticket de gestión por cada incidencia o notificación registrada.
- **Entradas:** Incidencia o notificación validada.
- **Salida:** Ticket generado y almacenado en el sistema.
- **Posibles errores:** Fallo en la generación del ticket.
- **Método de verificación:** Prueba funcional de creación automática de tickets.

3.2.6.21. Requisito funcional 45: "*Visualización de tickets en el panel de administración*"

- **Descripción:** El sistema deberá mostrar los tickets generados en el panel de administración para su seguimiento y resolución.
- **Entradas:** Tickets existentes en el sistema.
- **Salida:** Lista de tickets visible en el panel de administración.
- **Posibles errores:** Error al mostrar los tickets, e Inconsistencia en el estado de los tickets.
- **Método de verificación:** Prueba funcional de listado y actualización de tickets.

3.2.6.22. Requisito funcional 46: "*Almacenamiento del historial de rutas*"

- **Descripción:** La aplicación deberá almacenar en el historial del usuario las rutas finalizadas, conforme a la política de retención definida.
- **Entradas:** Ruta finalizada por el usuario.
- **Salida:** Historial individual de rutas almacenado.
- **Posibles errores:** Fallo al guardar la ruta en el historial.
- **Método de verificación:** Prueba funcional de persistencia del historial de rutas.

3.2.6.23. Requisito funcional 47: "*Interfaz de interacción con el usuario*"

- **Descripción:** La aplicación deberá proporcionar una interfaz gráfica que permita al usuario interactuar con las funcionalidades del sistema.
- **Entradas:** Acciones de interacción del usuario.
- **Salida:** Ejecución de las acciones correspondientes.
- **Posibles errores:** Fallo en la ejecución de acciones desde la interfaz.
- **Método de verificación:** Inspección y prueba funcional de la interfaz.

3.2.6.24. Requisito funcional 48: "*Botones de emergencia*"

- **Descripción:** La aplicación deberá incluir botones de emergencia accesibles desde la interfaz de usuario.
- **Entradas:** Activación del botón de emergencia por el usuario.
- **Salida:** Acción de emergencia ejecutada por el sistema.
- **Posibles errores:** El botón no ejecuta la acción asociada.
- **Método de verificación:** Prueba funcional de activación de botones de emergencia.

3.2.6.25. Requisito funcional 49: "*Botones de contacto rápido*"

- **Descripción:** La aplicación deberá incluir botones de contacto rápido que permitan al usuario iniciar acciones de contacto de forma inmediata.
- **Entradas:** Activación del botón de contacto rápido.
- **Salida:** Acción de contacto iniciada correctamente.
- **Posibles errores:** El botón no ejecuta la acción esperada.
- **Método de verificación:** Prueba funcional de botones de contacto rápido.

3.2.6.26. Requisito funcional 50: "*Uso de botones en situaciones de estrés*"

- **Descripción:** La aplicación deberá permitir la activación de los botones de emergencia y contacto rápido mediante una única acción del usuario, sin requerir confirmaciones adicionales.
- **Entradas:** Activación del botón correspondiente.
- **Salida:** Ejecución inmediata de la acción asociada al botón.
- **Posibles errores:** Requerir más de una interacción para ejecutar la acción, y No ejecución inmediata de la acción.
- **Método de verificación:** Prueba funcional de activación directa del botón.

3.2.6.27. Requisito funcional 51: "*Selección del idioma de la aplicación*"

- **Descripción:** La aplicación deberá permitir al usuario seleccionar el idioma de preferencia para la interfaz.
- **Entradas:** Preferencia de idioma del usuario.
- **Salida:** Interfaz mostrada en el idioma seleccionado.
- **Posibles errores:** El idioma seleccionado no se aplica correctamente.
- **Método de verificación:** Prueba funcional de cambio y persistencia de idioma.

3.2.6.28. Requisito funcional 52: "*Configuración de la posición de botones de emergencia*"

- **Descripción:** La aplicación deberá permitir al usuario configurar la posición de los botones de emergencia desde un menú de ajustes.
- **Entradas:** Preferencia de posición de los botones.
- **Salida:** Botones de emergencia colocados según la configuración del usuario.
- **Posibles errores:** No se guarda la configuración, y La posición configurada no se aplica correctamente.
- **Método de verificación:** Prueba funcional de configuración y persistencia de posiciones.

3.2.6.29. Requisito funcional 53: "*Solicitud de nuevo voluntario tras cancelación*"

- **Descripción:** La aplicación deberá permitir al usuario solicitar un nuevo voluntario cuando un acompañamiento haya sido cancelado por el voluntario asignado.
- **Entradas:** Cancelación del acompañamiento, y Solicitud de nuevo acompañamiento.
- **Salida:** Nueva solicitud de acompañamiento publicada.
- **Posibles errores:** No publicación de la nueva solicitud.
- **Método de verificación:** Prueba funcional de cancelación y nueva publicación.

3.2.6.30. Requisito funcional 54: "*Indicación del punto de inicio del acompañamiento*"

- **Descripción:** La aplicación deberá permitir al usuario indicar el punto de inicio al realizar una solicitud de acompañamiento.
- **Entradas:** Punto de inicio seleccionado.
- **Salida:** Solicitud de acompañamiento con punto de inicio incluido.
- **Posibles errores:** La solicitud no incluye el punto de inicio.
- **Método de verificación:** Prueba funcional de creación de solicitud con origen.

3.2.6.31. Requisito funcional 55: "*Indicación del punto final del acompañamiento*"

- **Descripción:** La aplicación deberá permitir al usuario indicar el punto final al realizar una solicitud de acompañamiento.
- **Entradas:** Punto final seleccionado.
- **Salida:** Solicitud de acompañamiento con punto final incluido.
- **Posibles errores:** La solicitud no incluye el punto final.
- **Método de verificación:** Prueba funcional de creación de solicitud con destino.

3.2.6.32. Requisito funcional 56: "*Visualización de la foto del voluntario*"

- **Descripción:** La aplicación deberá mostrar al usuario la foto de perfil del voluntario cuando este acepte una solicitud de acompañamiento.
- **Entradas:** Aceptación del acompañamiento por parte del voluntario.
- **Salida:** Foto de perfil del voluntario visible para el usuario.
- **Posibles errores:** Foto de perfil no disponible o no mostrada.
- **Método de verificación:** Prueba funcional de visualización de perfil del voluntario.

3.2.6.33. Requisito funcional 57: "*Visualización del nombre del voluntario*"

- **Descripción:** La aplicación deberá mostrar al usuario el nombre del voluntario asignado cuando este acepte una solicitud de acompañamiento.
- **Entradas:** Aceptación del acompañamiento por parte del voluntario.
- **Salida:** Nombre del voluntario visible para el usuario.
- **Posibles errores:** Nombre del voluntario no mostrado.
- **Método de verificación:** Prueba funcional de visualización del nombre del voluntario.

3.2.6.34. Requisito funcional 58: "*Visualización de información adicional del voluntario*"

- **Descripción:** La aplicación deberá mostrar información adicional del voluntario asignado cuando este acepte una solicitud de acompañamiento.
- **Entradas:** Aceptación del acompañamiento por parte del voluntario.
- **Salida:** Información adicional del voluntario mostrada al usuario.
- **Posibles errores:** Información del voluntario incompleta o no mostrada.
- **Método de verificación:** Inspección y prueba funcional de la información mostrada.

3.2.6.35. Requisito funcional 59: "*Confirmación de llegada segura al destino*"

- **Descripción:** La aplicación deberá permitir al usuario confirmar que ha llegado correctamente a su destino al finalizar el proyecto.
- **Entradas:** Confirmación del usuario al llegar al destino.
- **Salida:** Confirmación de llegada registrada en el sistema.
- **Posibles errores:** No se registra la confirmación del usuario.
- **Método de verificación:** Prueba funcional de confirmación de finalización de trayecto.

3.2.6.36. Requisito funcional 60: "*Visualización del mapa en la página principal*"

- **Descripción:** La aplicación deberá mostrar un mapa interactivo en la página principal que represente la posición del usuario y el entorno del campus.
- **Entradas:** Datos de cartografía, y Posición del usuario.
- **Salida:** Mapa interactivo visible para el usuario.
- **Posibles errores:** El mapa no puede cargarse o no permite interacción.
- **Método de verificación:** Prueba funcional de carga e interacción con el mapa.

3.2.6.37. Requisito funcional 61: "*Acceso a los detalles del perfil desde la página principal*"

- **Descripción:** La aplicación deberá proporcionar un botón en la página principal que permita acceder a la pantalla de detalles del perfil de usuario.
- **Entradas:** Activación del botón de perfil.
- **Salida:** Navegación a la pantalla de detalles del perfil.
- **Posibles errores:** El botón no redirige correctamente.
- **Método de verificación:** Prueba funcional de navegación entre pantallas.

3.2.6.38. Requisito funcional 62: "*Inicio de una ruta desde la página principal*"

- **Descripción:** La aplicación deberá permitir iniciar una ruta desde la página principal una vez definidos el origen y el destino.
- **Entradas:** Activación del botón de iniciar ruta.
- **Salida:** Inicio del cálculo y guiado de la ruta seleccionada.
- **Posibles errores:** El origen o destino no están definidos, y La ruta no se inicia correctamente.
- **Método de verificación:** Prueba funcional de inicio de ruta válida.

3.2.6.39. Requisito funcional 63: "*Visualización de información y estadísticas del usuario*"

- **Descripción:** La aplicación deberá mostrar, en la pantalla de detalles del perfil, la información introducida por el usuario y sus estadísticas asociadas.
- **Entradas:** Datos del perfil de usuario, y Datos estadísticos del usuario.
- **Salida:** Información personal y estadísticas visibles.
- **Posibles errores:** Estadísticas desactualizadas o no disponibles.

- **Método de verificación:** Prueba funcional de visualización de datos y estadísticas.

3.2.6.40. Requisito funcional 64: "*Indicador de información pendiente en el perfil*"

- **Descripción:** La aplicación deberá indicar al usuario qué información de su perfil está pendiente de completar.
- **Entradas:** Información actual del perfil del usuario.
- **Salida:** Lista o indicador de información pendiente.
- **Posibles errores:** No indica correctamente la información faltante.
- **Método de verificación:** Prueba funcional de detección de campos incompletos.

3.2.6.41. Requisito funcional 65: "*Restricción de correos electrónicos institucionales*"

- **Descripción:** La aplicación deberá permitir el registro y acceso únicamente mediante direcciones de correo institucionales de la URJC.
- **Entradas:** Dirección de correo electrónico del usuario.
- **Salida:** Aceptación de correos válidos, y Rechazo de correos no institucionales.
- **Posibles errores:** Correo electrónico fuera del dominio permitido.
- **Método de verificación:** Prueba funcional de validación de dominio de correo.

3.2.7. Datos usuario

3.2.7.1. Requisito funcional 66 : "*Actualización del perfil de usuario*"

- **Descripción:** La aplicación deberá permitir al usuario completar y actualizar la información de su perfil.
- **Entradas:** Información del perfil proporcionada por el usuario.
- **Salida:** Perfil de usuario actualizado.
- **Posibles errores:** Datos introducidos inválidos o inconsistentes.
- **Método de verificación:** Prueba funcional de edición y guardado del perfil

3.2.7.2. Requisito funcional 67: "*Gestión de la foto de perfil*"

- **Descripción:** La aplicación deberá permitir al usuario añadir o modificar su foto de perfil.
- **Entradas:** Imagen seleccionada como foto de perfil.
- **Salida:** Perfil actualizado con la foto de perfil.
- **Posibles errores:** Formato de imagen no válido, y Error al cargar la imagen.
- **Método de verificación:** Prueba funcional de carga y visualización de imagen.

3.2.7.3. Requisito funcional 68: "*Indicación del género en el perfil*"

- **Descripción:** La aplicación deberá permitir al usuario indicar su género dentro de la información de su perfil.
- **Entradas:** Género seleccionado por el usuario.
- **Salida:** Perfil actualizado con el género indicado.
- **Posibles errores:** Valor de género no válido.
- **Método de verificación:** Prueba funcional de almacenamiento del atributo de género.

3.2.7.4. Requisito funcional 69: "*Edición de información no fundamental del perfil*"

- **Descripción:** La aplicación deberá permitir al usuario modificar la información no fundamental de su perfil en cualquier momento.
- **Entradas:** Nueva información proporcionada por el usuario.
- **Salida:** Perfil actualizado con la información modificada.
- **Posibles errores:** Error en la actualización de la información.
- **Método de verificación:** Prueba funcional de modificación de datos del perfil.

3.2.7.5. Requisito funcional 70: "*Consulta del historial de rutas del usuario*"

- **Descripción:** La aplicación deberá permitir al usuario consultar el historial de sus rutas realizadas.
- **Entradas:** Solicitud de consulta del historial.
- **Salida:** Lista de rutas realizadas por el usuario.
- **Posibles errores:** No existen rutas almacenadas para el usuario, y Error en la recuperación del historial.
- **Método de verificación:** Prueba funcional de consulta del historial de rutas.

3.2.7.6. Requisito funcional 71: "*Visualización de estadísticas del usuario*"

- **Descripción:** La aplicación deberá mostrar al usuario estadísticas agregadas basadas en las rutas realizadas.
- **Entradas:** Datos históricos de rutas del usuario.
- **Salida:** Estadísticas del usuario mostradas en la aplicación.
- **Posibles errores:** Datos insuficientes para generar estadísticas, y Error al calcular estadísticas.
- **Método de verificación:** Prueba funcional de generación y visualización de estadísticas.

3.2.7.7. Requisito funcional 72: "*Configuración de calibraciones de rutas*"

- **Descripción:** La aplicación deberá proporcionar un apartado de calibraciones que permita al usuario configurar preferencias para el cálculo de rutas.
- **Entradas:** Preferencias de calibración del usuario.
- **Salida:** Preferencias de calibración almacenadas.
- **Posibles errores:** No se guardan las preferencias de calibración, y Preferencias no aplicadas al cálculo de rutas.
- **Método de verificación:** Prueba funcional de configuración y persistencia de calibraciones.

3.2.7.8. Requisito funcional 73: "*Calibración de condiciones de iluminación*"

- **Descripción:** La aplicación deberá permitir al usuario definir preferencias relacionadas con el nivel de iluminación en el cálculo de rutas.
- **Entradas:** Preferencias de iluminación del usuario.
- **Salida:** Preferencias de iluminación registradas.
- **Posibles errores:** Preferencias ignoradas por el algoritmo de rutas.
- **Método de verificación:** Prueba funcional del impacto de la calibración de iluminación en el enrutado.

3.2.7.9. Requisito funcional 74: "*Calibración de condiciones de afluencia*"

- **Descripción:** La aplicación deberá permitir al usuario definir preferencias relacionadas con la afluencia de personas en el cálculo de rutas.
- **Entradas:** Preferencias de afluencia del usuario.
- **Salida:** Preferencias de afluencia registradas.
- **Posibles errores:** Preferencias ignoradas por el algoritmo de rutas.
- **Método de verificación:** Prueba funcional del impacto de la calibración de afluencia en el cálculo de rutas.

3.2.7.10. Requisito funcional 75: "*Calibración del estado entorno*"

- **Descripción:** La aplicación deberá permitir al usuario definir preferencias relacionadas con el estado del entorno en el cálculo de rutas.
- **Entradas:** Preferencias del usuario sobre el estado del entorno.
- **Salida:** Preferencias del estado del entorno almacenadas.
- **Posibles errores:** Preferencias no consideradas en el algoritmo de rutas.
- **Método de verificación:** Prueba funcional de aplicación de preferencias del entorno.

3.2.7.11. Requisito funcional 76: "*Sistema de puntuación del acompañado*"

- **Descripción:** La aplicación deberá permitir registrar y mostrar la puntuación media del usuario cuando actúa como acompañado.
- **Entradas:** Puntuaciones recibidas como acompañado.
- **Salida:** Estadística de puntuación del perfil como acompañado.
- **Posibles errores:** Cálculo incorrecto de la puntuación media.
- **Método de verificación:** Prueba funcional de cálculo y visualización de puntuaciones.

3.2.7.12. Requisito funcional 77: "*Sistema de puntuación del voluntario*"

- **Descripción:** La aplicación deberá permitir registrar y mostrar la puntuación media del usuario cuando actúa como voluntario.
- **Entradas:** Puntuaciones recibidas como voluntario.
- **Salida:** Estadística de puntuación del perfil como voluntario.
- **Posibles errores:** Cálculo incorrecto de la puntuación media.
- **Método de verificación:** Prueba funcional de cálculo y visualización de puntuaciones.

3.2.8. Voluntarios

3.2.8.1. Requisito funcional 78: "*Visualización de solicitudes de acompañamiento*"

- **Descripción:** La aplicación deberá permitir a los voluntarios visualizar las solicitudes de acompañamiento disponibles.
- **Entradas:** Solicitudes.
- **Salida:** Solicitud aceptada.
- **Posibles errores:** No ver las solicitudes.
- **Método de verificación:** P

3.2.8.2. Requisito funcional 79: "*Respuesta a solicitudes de acompañamiento*"

- **Descripción:** La aplicación deberá permitir a los voluntarios aceptar o rechazar solicitudes de acompañamiento.
- **Entradas:** Solicitud seleccionada, y Respuesta del voluntario.
- **Salida:** Solicitud marcada como aceptada o rechazada.
- **Posibles errores:** No se registra la respuesta del voluntario.
- **Método de verificación:** Prueba funcional de aceptación y rechazo de solicitudes.

3.2.8.3. Requisito funcional 80: "*Registro y gestión de voluntarios*"

- **Descripción:** La aplicación deberá mantener un sistema de gestión de usuarios registrados como voluntarios.
- **Entradas:** Solicitudes de alta como voluntario.
- **Salida:** Lista actualizada de voluntarios disponibles.
- **Posibles errores:** Error en el registro como voluntario, y Lista de voluntarios incompleta o inconsistente.
- **Método de verificación:** Prueba funcional de alta y consulta de voluntarios.

3.2.8.4. Requisito funcional 81: "*Alta como voluntario*"

- **Descripción:** La aplicación deberá permitir a un usuario registrarse como voluntario para ofrecer acompañamientos.
- **Entradas:** Solicitud de alta como voluntario.
- **Salida:** Confirmación del alta como voluntario.
- **Posibles errores:** Fallo al registrar la solicitud, y Usuario no autorizado para ser voluntario.
- **Método de verificación:** Prueba funcional de alta como voluntario.

3.2.8.5. Requisito funcional 82: "*Solicitud de acompañamiento*"

- **Descripción:** La aplicación deberá permitir al usuario solicitar un acompañamiento seleccionando entre los voluntarios disponibles.
- **Entradas:** Solicitud de acompañamiento del usuario.
- **Salida:** Solicitud de acompañamiento registrada y publicada.
- **Posibles errores:** No existen voluntarios disponibles, y Error al registrar la solicitud.
- **Método de verificación:** Prueba funcional de creación de solicitudes de acompañamiento.

3.2.8.6. Requisito funcional 83: "*Activación del modo "Voy Contigo""*

- **Descripción:** La aplicación deberá permitir al usuario activar el modo "Voy Contigo" para compartir su trayecto con un contacto de confianza o solicitar acompañamiento.
- **Entradas:** Activación explícita del modo por el usuario.
- **Salida:** Modo "Voy Contigo" activado para el trayecto actual.
- **Posibles errores:** Activación accidental del modo, y Fallo al activar el modo.
- **Método de verificación:** Prueba funcional de activación y desactivación controlada.

3.2.8.7. Requisito funcional 84: "*Visualización del punto de inicio del acompañamiento*"

- **Descripción:** La aplicación deberá mostrar al voluntario el punto de inicio del acompañamiento en las solicitudes recibidas.
- **Entradas:** Solicitud de acompañamiento con datos.
- **Salida:** Punto de inicio visible para el voluntario.
- **Posibles errores:** El punto de inicio no está incluido en la solicitud.
- **Método de verificación:** Prueba funcional de visualización del origen del acompañamiento.

3.2.8.8. Requisito funcional 85: "*Visualización del punto final del acompañamiento*"

- **Descripción:** La aplicación deberá mostrar al voluntario el punto final del acompañamiento en las solicitudes recibidas.
- **Entradas:** Solicitud de acompañamiento con datos.
- **Salida:** Punto final visible para el voluntario.
- **Posibles errores:** El punto final no está incluido en la solicitud.
- **Método de verificación:** Prueba funcional de visualización del destino del acompañamiento.

3.2.8.9. Requisito funcional 86: "*Visualización del nombre del solicitante*"

- **Descripción:** La aplicación deberá mostrar al voluntario el nombre del usuario solicitante cuando acepte una solicitud de acompañamiento.
- **Entradas:** Solicitud aceptada por el voluntario.
- **Salida:** Nombre del solicitante visible para el voluntario.
- **Posibles errores:** El nombre del solicitante no está disponible.
- **Método de verificación:** Prueba funcional de visualización de identidad del solicitante.

3.2.8.10. Requisito funcional 87: "*Visualización de la foto del solicitante*"

- **Descripción:** La aplicación deberá mostrar al voluntario la foto de perfil del usuario solicitante tras aceptar el acompañamiento.
- **Entradas:** Solicitud aceptada.
- **Salida:** Foto de perfil del solicitante visible.
- **Posibles errores:** Foto de perfil no disponible, y Error al mostrar la imagen.
- **Método de verificación:** Prueba funcional de visualización de la foto del solicitante.

3.2.8.11. Requisito funcional 88: "*Visualización de información adicional del solicitante*"

- **Descripción:** La aplicación deberá mostrar al voluntario información adicional del perfil del usuario solicitante tras aceptar el acompañamiento.
- **Entradas:** Solicitud aceptada.
- **Salida:** Información adicional del solicitante visible.
- **Posibles errores:** Información incompleta o no disponible.
- **Método de verificación:** Inspección y prueba funcional de información mostrada.

3.2.8.12. Requisito funcional 89: "*Cancelación de acompañamiento por el voluntario*"

- **Descripción:** La aplicación deberá permitir al voluntario cancelar un acompañamiento previamente aceptado.
- **Entradas:** Solicitud de cancelación del voluntario.
- **Salida:** Acompañamiento cancelado, y Solicitud retirada del listado del voluntario.
- **Posibles errores:** Cancelación elimina la solicitud para todos los usuarios, y Error al cancelar el acompañamiento.

- **Método de verificación:** Prueba funcional de cancelación controlada.

3.2.8.13. Requisito funcional 90: *"Visualización de la hora prevista del acompañamiento"*

- **Descripción:** La aplicación deberá mostrar al voluntario la hora prevista de inicio del acompañamiento en cada solicitud.
- **Entradas:** Solicitud de acompañamiento.
- **Salida:** Hora prevista de inicio visible para el voluntario.
- **Posibles errores:** La solicitud no incluye la hora de inicio.
- **Método de verificación:** Prueba funcional de visualización temporal del acompañamiento.

3.2.8.14. Requisito funcional 91: *"Puntuación del acompañado"*

- **Descripción:** La aplicación deberá permitir al voluntario puntuar al usuario acompañado al finalizar un acompañamiento.
- **Entradas:** Puntuación asignada por el voluntario.
- **Salida:** Puntuación almacenada para el cálculo de estadísticas del acompañado.
- **Posibles errores:** Fallo al almacenar la puntuación.
- **Método de verificación:** Prueba funcional de registro y consulta de puntuaciones.

3.2.8.15. Requisito funcional 92: *"Puntuación del voluntario"*

- **Descripción:** La aplicación deberá permitir al usuario acompañado puntuar al voluntario al finalizar un trayecto acompañado.
- **Entradas:** Puntuación asignada por el usuario.
- **Salida:** Puntuación almacenada para el cálculo de estadísticas del voluntario.
- **Posibles errores:** Fallo al almacenar la puntuación.
- **Método de verificación:** Prueba funcional de registro y consulta de puntuaciones.

3.2.8.16. Requisito funcional 93: *"Confirmación de llegada del voluntario"*

- **Descripción:** La aplicación deberá solicitar al voluntario la confirmación de que ha llegado correctamente al destino al finalizar el acompañamiento.
- **Entradas:** Solicitud de confirmación enviada al voluntario.
- **Salida:** Confirmación de llegada registrada.
- **Posibles errores:** No se recibe la confirmación del voluntario.
- **Método de verificación:** Prueba funcional de solicitud y registro de confirmación.

3.2.9. Seguridad en los caminos

3.2.9.1. Requisito funcional 94: *"Seguimiento del estado de seguridad durante el trayecto"*

- **Descripción:** La aplicación deberá supervisar el estado de seguridad del usuario durante un trayecto activo mediante el análisis de su posición.
- **Entradas:** Posición del usuario durante el trayecto.
- **Salida:** Estado de seguridad evaluado para el usuario.

- **Posibles errores:** No es posible evaluar el estado de seguridad.
- **Método de verificación:** Prueba funcional de supervisión continua del trayecto.

3.2.9.2. Requisito funcional 95: "*Detección de detención durante el trayecto*"

- **Descripción:** La aplicación deberá detectar si el usuario se detiene de forma prolongada durante un trayecto activo.
- **Entradas:** Posición del usuario a lo largo del tiempo.
- **Salida:** Estado de movimiento del usuario (en movimiento/detenido).
- **Posibles errores:** No se puede determinar el movimiento del usuario.
- **Método de verificación:** Prueba funcional de detección de detenciones.

3.2.9.3. Requisito funcional 96: "*Detección de desvío de la ruta*"

- **Descripción:** La aplicación deberá detectar si el usuario se desvía significativamente de la ruta activa durante el trayecto.
- **Entradas:** Posición del usuario, y Ruta activa.
- **Salida:** Estado de seguimiento de la ruta(en ruta/desviado).
- **Posibles errores:** No se puede determinar el seguimiento de la ruta.
- **Método de verificación:** Prueba funcional de detección de desvíos.

3.2.9.4. Requisito funcional 97: "*Emisión de prealerta*"

- **Descripción:** La aplicación deberá emitir una prealerta cuando detecte una situación anómala durante el trayecto (detención prolongada o desvío de ruta).
- **Entradas:** Evento de estado anómalo.
- **Salida:** Prealerta enviada al usuario.
- **Posibles errores:** Fallo en la emisión de la prealerta.
- **Método de verificación:** Prueba funcional de emisión de prealertas.

3.2.9.5. Requisito funcional 98: "*Solicitud de confirmación de estado al usuario*"

- **Descripción:** La aplicación deberá solicitar al usuario que confirme si se encuentra bien tras la emisión de una prealerta.
- **Entradas:** Prealerta activada.
- **Salida:** Solicitud de confirmación enviada al usuario.
- **Posibles errores:** Fallo al enviar la solicitud de confirmación.
- **Método de verificación:** Prueba funcional de envío de solicitud de confirmación.

3.2.9.6. Requisito funcional 99: "*Detección de ausencia de respuesta del usuario*"

- **Descripción:** La aplicación deberá detectar que el usuario no ha respondido a la solicitud de confirmación de estado.
- **Entradas:** Solicitud de confirmación enviada.
- **Salida:** Estado de solicitud sin respuesta.
- **Posibles errores:** No es posible determinar el estado de la respuesta.
- **Método de verificación:** Prueba funcional de detección de falta de respuesta.

3.2.9.7. Requisito funcional 100: "*Detección de ausencia de respuesta tras 30 segundos*"

- **Descripción:** La aplicación deberá detectar que han transcurrido 30 segundos desde la solicitud de confirmación sin recibir respuesta del usuario.
- **Entradas:** Solicitud de confirmación, y Marca temporal de envío.
- **Salida:** Confirmación de tiempo de espera superado.
- **Posibles errores:** No se puede determinar el tiempo transcurrido.
- **Método de verificación:** Prueba funcional de temporización de 30 segundos.

3.2.9.8. Requisito funcional 101: "*Envío de alerta al contacto de confianza*"

- **Descripción:** La aplicación deberá enviar una alerta de incidencia al contacto de confianza configurado cuando el usuario no responda tras el tiempo establecido.
- **Entradas:** Confirmación de ausencia de respuesta.
- **Salida:** Alerta enviada al contacto de confianza.
- **Posibles errores:** No es posible contactar con el contacto de confianza.
- **Método de verificación:** Prueba funcional de envío de alertas a contactos.

3.2.9.9. Requisito funcional 101: "*Envío de alerta al servicio de seguridad del campus*"

- **Descripción:** La aplicación deberá enviar una alerta de incidencia al servicio de seguridad del campus cuando el usuario no responda tras el tiempo establecido.
- **Entradas:** Confirmación de ausencia de respuesta.
- **Salida:** Alerta enviada al servicio de seguridad del campus.
- **Posibles errores:** No es posible contactar con el servicio de seguridad.
- **Método de verificación:** Prueba funcional de envío de alertas al servicio de seguridad.

3.2.9.10. Requisito funcional 102: "*Inclusión de la última ubicación conocida en alertas de incidencia*"

- **Descripción:** La aplicación deberá incluir la última ubicación conocida del usuario en las alertas de incidencia enviadas a terceros.
- **Entradas:** Última ubicación conocida del usuario, y Confirmación de ausencia de respuesta del usuario.
- **Salida:** Alerta de incidencia que incluye la última ubicación conocida.
- **Posibles errores:** No se dispone de una ubicación válida del usuario.
- **Método de verificación:** Prueba funcional de generación de alertas con datos de ubicación.

3.2.9.11. Requisito funcional 103: "*Configuración del tiempo de espera para alertas*"

- **Descripción:** El sistema deberá permitir a los administradores configurar el tiempo de espera tras el cual se emite una alerta de incidencia.
- **Entradas:** Nuevo valor de tiempo de espera configurado por un administrador.
- **Salida:** Tiempo de espera para alertas actualizado en el sistema.
- **Posibles errores:** El nuevo tiempo de espera no se guarda o no se aplica correctamente.
- **Método de verificación:** Prueba funcional de configuración y aplicación del tiempo de espera.

3.2.10. Incidencias

3.2.10.1. Requisito funcional 104: "*Reporte de incidencias por el usuario*"

- **Descripción:** La aplicación deberá permitir a los usuarios reportar incidencias relacionadas con la seguridad y el estado del entorno.
- **Entradas:** Datos del reporte de incidencia.
- **Salida:** Incidencia registrada en el sistema.
- **Posibles errores:** Error al registrar la incidencia, y Datos de incidencia incompletos o inválidos.
- **Método de verificación:** Prueba funcional de creación y almacenamiento de incidencias.

3.2.10.2. Requisito funcional 105: "*Reporte de farolas fundidas*"

- **Descripción:** La aplicación deberá permitir a los usuarios reportar incidencias de farolas fundidas.
- **Entradas:** Reporte de farola fundida.
- **Salida:** Incidencia de farola registrada en el sistema.
- **Posibles errores:** Error en el registro del reporte, y Ubicación de la farola no válida.
- **Método de verificación:** Prueba funcional de reporte de incidencias de alumbrado.

3.2.10.3. Requisito funcional 106: "*Reporte de zonas solitarias*"

- **Descripción:** La aplicación deberá permitir a los usuarios reportar zonas percibidas como solitarias o inseguras.
- **Entradas:** Reporte de zona solitaria.
- **Salida:** Incidencia de zona solitaria registrada en el sistema.
- **Posibles errores:** Error en el registro del reporte.
- **Método de verificación:** Prueba funcional de reporte de zonas solitarias.

3.2.10.4. Requisito funcional 107: *"Reporte de obstáculos en la vía"*

- **Descripción:** La aplicación deberá permitir a los usuarios reportar obstáculos presentes en la vía que puedan afectar a la seguridad del trayecto.
- **Entradas:** Reporte de obstáculo en la vía.
- **Salida:** Incidencia de obstáculo registrada en el sistema.
- **Posibles errores:** Error en el registro del reporte, y Ubicación del obstáculo no válida.
- **Método de verificación:** Prueba funcional de reporte de obstáculos.

3.2.10.5. Requisito funcional 108: *"Ajuste del nivel de seguridad de rutas afectadas por incidencias"*

- **Descripción:** La aplicación deberá ajustar el nivel de seguridad de las rutas y zonas afectadas por las incidencias reportadas.
- **Entradas:** Resultados del algoritmo de rutas, e Incidencias asociadas a rutas o zonas.
- **Salida:** Nuevos niveles de seguridad aplicados a rutas y zonas.
- **Posibles errores:** Ajustes incorrectos del nivel de seguridad, e Inconsistencia entre incidencias y rutas afectadas.
- **Método de verificación:** Prueba funcional de recalibración del nivel de seguridad.

3.2.10.6. Requisito funcional 109: *"Envío de reconfirmación al usuario"*

- **Descripción:** La aplicación deberá enviar una solicitud de reconfirmación al usuario cuando se haya generado una incidencia y haya transcurrido el tiempo de espera configurado.
- **Entradas:** Ausencia prolongada de respuesta del usuario.
- **Salida:** Solicitud de reconfirmación enviada al usuario.
- **Posibles errores:** Fallo al enviar la reconfirmación.
- **Método de verificación:** Prueba funcional de temporización y envío de reconfirmación.

3.2.10.7. Requisito funcional 110: *"Respuesta del usuario a la reconfirmación"*

- **Descripción:** La aplicación deberá permitir al usuario responder a la solicitud de reconfirmación indicando su situación.
- **Entradas:** Solicitud de reconfirmación recibida.
- **Salida:** Respuesta del usuario registrada en el sistema.
- **Posibles errores:** No se registra la respuesta del usuario.
- **Método de verificación:** Prueba funcional de recepción de respuesta.

3.2.10.8. Requisito funcional 111: "*Declaración de incidencia grave*"

- **Descripción:** La aplicación deberá declarar una incidencia grave cuando el usuario no responda a la reconfirmación tras el tiempo máximo establecido.
- **Entradas:** Ausencia prolongada de respuesta tras reconfirmación.
- **Salida:** Incidencia marcada como grave en el sistema.
- **Posibles errores:** No se detecta correctamente la ausencia prolongada.
- **Método de verificación:** Prueba funcional de escalado de incidencias.

3.2.10.9. Requisito funcional 112: "*Incremento de medidas ante incidencia grave*"

- **Descripción:** La aplicación deberá aplicar medidas de seguridad adicionales cuando se declare una incidencia grave.
- **Entradas:** Incidencia marcada como grave.
- **Salida:** Medidas de seguridad ampliadas activadas.
- **Posibles errores:** Medidas adicionales no aplicadas.
- **Método de verificación:** Prueba funcional de activación de medidas de emergencia.

3.3 Requisitos no funcionales

3.3.1 Rendimiento

3.3.1.1 RNF-1: "*Tiempo de respuesta del cálculo de rutas*"

El sistema deberá mostrar las rutas disponibles en un tiempo máximo de 3 segundos desde la solicitud del usuario.

3.3.1.2 RNF-2: "*Actualización de incidencias*"

El sistema deberá actualizar la información de incidencias recibidas en un tiempo máximo de 10 segundos.

3.3.2 Disponibilidad

3.3.2.1 RNF-3: "*Servicio continuo*"

La aplicación deberá estar disponible 24 horas al día, los 7 días de la semana.

3.3.2.2 RNF-4: "*Alta disponibilidad*"

El sistema deberá garantizar la continuidad del servicio mediante mecanismo de alta disponibilidad.

3.3.2.3 RNF-5: "*Sistema de respaldo*"

El sistema deberá garantizar la continuidad del servicio mediante mecanismos de alta disponibilidad.

3.3.3 Seguridad

3.3.3.1 RNF-6: *"Comunicación segura"*

La comunicación deberá realizarse mediante HTTPS.

3.3.3.2 RNF-7: *"Acceso mediante SSO URJC"*

El acceso deberá realizarse exclusivamente mediante el SSO de la URJC.

3.3.4 Privacidad

3.3.4.1 RNF-8: *"Protección de datos"*

Los datos deberán protegerse frente a accesos o modificaciones no autorizadas.

3.3.4.2 RNF-9: *"Retención del historial de rutas"*

El historial deberá almacenarse un máximo de 24 horas, salvo incidencias activas.

3.3.4.3 RNF-10: *"Historial con incidencias"*

Las rutas con incidencias deberán conservarse hasta la resolución de la incidencia.

3.3.4.4 RNF-11: *"Anonimización de estadísticas"*

Las estadísticas deberán mostrarse de forma anonimizada.

3.3.5 Usabilidad y accesibilidad

3.3.5.1 RNF-12: *"Botones accesibles"*

Los botones de emergencia deberán ser accesibles con una sola mano y sin atención visual.

3.3.5.2 RNF-13: *"Accesibilidad universal"*

La interfaz deberá ser accesible para personas con diversidad funcional.

3.3.5.3 RNF-14 : *"Uso en situaciones de estrés"*

La aplicación deberá permitir el acceso rápido a funciones críticas.

3.3.6 Compatibilidad

3.3.6.1 RNF-15: *"Android e iOS"*

La aplicación deberá estar disponible en Android e iOS.

3.3.7 Idiomas

3.3.7.1 RNF-16: *"Idiomas soportados"*

La aplicación deberá estar disponible al menos en español e inglés.

3.3.8 Integración con otro sistema

3.3.8.1 RNF-17: *"Intercambio de información"*

Los datos con LumenSmart deberán intercambiarse en JSON UTF-8.

3.3.8.2 RNF-18: *"Identificación de sensores"*

Sensores y farolas deberán tener identificador único y ubicación asociada.

3.3.9 Configuración

3.3.9.1 RNF-19: "*Parámetros configurables*"

El sistema deberá permitir configurar parámetros operativos críticos.

3.3.10 Desarrollo

3.3.10.1 RNF-20: "Tecnologías abiertas"

Se deberá priorizar el uso de tecnologías de código abierto.

3.4 Requisitos de rendimiento

El consumo de memoria y espacio que ocupe en disco debe ser el menos posible, debido a que se busca una aplicación útil para el móvil.

El número de terminales soportados debe de ser como mínimo el número total de integrantes de la universidad.

Con el número total de usuarios simultáneos que soporta el sistema también ocurre lo mismo que en el caso anterior. Como mínimo el total deberá ser el número de personas que integran la universidad aunque sea muy poco probable que todas las personas usen la aplicación y la usen a la vez.

La aplicación no debe de consumir recursos excesivos, debe de ser rápida y funcionar de una manera fluida.

3.5 Requisitos de diseño

Podrá hacerse uso de la aplicación dispositivos Android e iOs. Esta estará limitada por las antiguas versiones de Android y los requisitos exigentes de la appstore.Habrá que cumplir que modelos antiguos de android con mayor limitación técnica en cuanto a potencia y memoria se refiere puedan utilizar la aplicación. Todo debe estar perfectamente documentado para así poder ser extendido en el futuro.

3.6 Atributos del sistema software

La aplicación quedará instalada en servidores de la URJC y será mantenida por el personal de informática. El mantenimiento de esta será el menor y más sencillo que sea posible.

La aplicación no debe fallar. Para garantizar una mínima seguridad, la contraseña no debe de poder ser expuesta en ningún momento.

Como ya se ha comentado anteriormente, la aplicación estará disponible para dispositivos Android y páginas web (aparte de su uso como aplicación de escritorio), de esta manera se garantiza su disponibilidad como su portabilidad.

La aplicación tiene que contener el menor número posible de errores, en el caso de que los contenga habrá un periodo de garantía.

4. Anexo

Actores	Descripción
Usuario	Nombre para el usuario genérico que engloba al resto de tipos de usuario
Usuario URJC	Usuario perteneciente a la URJC
Usuario PAS	Usuario del personal de administración de la universidad
Usuario externo	Usuario no perteneciente a la URJC
Servidor LDAP	Servidor que comprueba datos de inicio de sesión de los usuarios que pertenecen a la URJC

ID/Nombre:	CU01 – RegistrarUsuarioURJC	
Actores:	UsuarioURJC	
Objetivo:	Permitir a un usuario vinculado a la URJC registrarse en la aplicación.	
Descripción:	Permite el registro de usuarios que forman parte de la URJC. Se les solicitará sus datos personales para realizar dicho registro.	
Tipo:	Primario	
Precondiciones:	El usuario debe estar vinculado a la URJC y no debe estar registrado en la aplicación.	
Dependencias:	Ninguna	
Secuencia normal:	**N. º de paso**	**Acción**
	1	El usuario vinculado a la URJC que desee registrarse entrará en la aplicación.
	2	La aplicación pide al usuario su nombre completo, DNI, fecha de nacimiento, teléfono, correo y contraseña institucional para poder realizar el registro.
	3	Los servidores LDAP de la universidad comprueban que la cuenta de correo pertenezca a un usuario de la URJC

	4	La aplicación enviará un correo al usuario para que confirme el alta.
	5	El usuario confirma el alta.
	6	Se crea la cuenta
Secuencia alternativa:	**N. º de paso**	**Acción**
	3.1	Si el usuario ya se encuentra registrado, la aplicación mostrará un mensaje de error y no le permitirá acceder a la aplicación.
Postcondiciones:	La aplicación registra al usuario y sus datos se guardan en la base de datos.	
Importancia:	Alta	
Comentario:	Todo personal de la URJC puede registrarse en la aplicación, que almacenará información acerca del tipo de cuenta dependiendo si es Alumno, PAS o PDI.	

ID/Nombre:	CU-02 RegistrarUsuarioExterno	
Actores:	UsuarioExterno	
Objetivo:	Permitir a un usuario no vinculado a la URJC registrarse en la aplicación.	
Descripción:	Permite el registro de usuarios que no forman parte de la URJC. Se les solicitará que rellenen un formulario con sus datos personales para realizar dicho registro.	
Tipo:	Primaria	
Precondiciones:	El usuario no debe estar vinculado a la URJC ni registrado en la aplicación.	
Dependencias:	Ninguna	
Secuencia normal:	**N. º de paso**	**Acción**
	1	El usuario vinculado a la URJC que desee registrarse entrará en la aplicación.

	2	El usuario rellenará un formulario con el correo electrónico, nombre completo, DNI, fecha de nacimiento y teléfono móvil.
	3	El personal de la universidad validará la solicitud
	4	La aplicación envía un correo para que confirme el alta y establezca la contraseña.
	5	El usuario confirma el alta a través del correo que ha recibido y establece la contraseña.
	6	Se crea la cuenta.
Secuencia alternativa:	**N. º de paso**	**Acción**
	3.1	Si la cuenta ya existe se envía un correo informando que ya se registró antes.
Postcondiciones:	El usuario se ha registrado en la aplicación y se han guardado sus datos.	
Importancia:	Alta	
Comentario:	Un usuario no registrado externo a la URJC accede a la aplicación a través de la pestaña Externo, donde deberá rellenar un formulario con sus datos personales.	

ID/Nombre:	CU-03 ConfirmarAlta
Actores:	Servidor LDAP, Usuario PAS
Objetivo:	Confirmar el alta de registro de un nuevo usuario
Descripción:	La aplicación enviará un correo al usuario registrado para confirmar su alta en la aplicación.
Tipo:	Primaria
Precondiciones:	El usuario debe haber introducido sus datos personales para realizar el registro.

Dependencias:	CU- 01 RegistrarUsuarioURJC CU- 02 RegistrarUsuarioExterno	
Secuencia normal:	**N. º de paso**	**Acción**
	1	El usuario se registra introduciendo sus datos personales
	2	La aplicación manda un correo al usuario para que verifique su identidad.
	3	La aplicación recibe la confirmación del usuario y crea la cuenta.
Secuencia alternativa:	**N. º de paso**	**Acción**
	2	La dirección de correo de introducida es incorrecta y no se puede realizar el registro del usuario.
Postcondiciones:	El usuario registrado podrá iniciar sesión y acceder a la aplicación.	
Importancia:	Alta	
Comentario:	Ninguno	

ID/Nombre:	CU-04 Iniciar_Sesión
Actores:	Usuario
Objetivo:	El usuario puede introducir sus datos y acceder a la aplicación.
Descripción:	El usuario introduce su correo y contraseña e inicia sesión en la aplicación.
Tipo:	Primario
Precondiciones:	El usuario se ha registrado previamente en la aplicación.
Dependencias:	CU-01 RegistrarUsuarioExterno CU-02 RegistrarUsuarioURJC

	CU-03 ConfirmarAlta	
Secuencia normal:	**N. º de paso**	**Acción**
	1	El usuario entra en la aplicación.
	2	El usuario introduce sus correo y contraseña en la aplicación.
	3	El usuario accede a la página principal de la aplicación.
Secuencia alternativa:	**N. º de paso**	**Acción**
	2.1	El usuario introduce mal su contraseña tres veces y la aplicación bloquea al usuario.
Postcondiciones:	El usuario ha accedido correctamente a la aplicación y puede utilizarla.	
Importancia:	Alta	
Comentario:	Cada 15 días el Servidor URJC envía un SMS al teléfono móvil para que el usuario confirme su identidad al iniciar sesión.	

ID/Nombre:	CU-05 Verificar_Credenciales
Actores:	Servidor LDAP
Objetivo:	Cuando un usuario inicia sesión el servidor LDAP verifica que los datos introducidos son correctos.
Descripción:	Cuando un usuario inicia sesión introduciendo su contraseña y su correo el servidor LDAP verifica que dichos datos son correctos.
Tipo:	Primaria
Precondiciones:	El usuario que inicia sesión este previamente registrado
Dependencias:	CU-01 RegistrarUsuarioExterno

	CU-02 RegistrarUsuarioURJC CU-03 ConfirmarAlta	
Secuencia normal:	**N. º de paso**	**Acción**
	1	El usuario inicia sesión con su correo y contraseña
	2	El servidor LDAP verifica que los datos introducidos son correctos
	3	Si los datos son correctos el usuario podrá acceder a la aplicación
Secuencia alternativa:	**N. º de paso**	**Acción**
	3.1	Si los datos no son correctos se volverá a la pantalla de inicio de sesión donde el usuario tendrá 3 intentos para introducir los datos correctamente
Postcondiciones:	El usuario accede a la aplicación y podrá utilizar todas sus ventajas, como por ejemplo apuntarse a una actividad o alquilar una instalación	
Importancia:	Media/Alta	
Comentario:	Todo usuario registrado en la aplicación podrá iniciar sesión con su correo y contraseña, esos datos serán verificados por el servidor LDAP para comprobar que el usuario ha sido registrado previamente. Una vez verificados los datos, el usuario podrá acceder a la aplicación, pero, en caso de que los datos sean incorrectos dispondrá de 3 intentos para introducirlos correctamente o si no será bloqueado	

ID/Nombre:	CU-06 Bloquear_Usuario
Actores:	Ninguno
Objetivo:	Bloquear a los usuarios que al iniciar sesión superen los 3 intentos
Descripción:	Tras comprobar el número de intentos de inicio de sesión, al ser superior a 3 se procede a bloquear el usuario para que este restablezca su contraseña.

Tipo:	Opcional	
Precondiciones:	Estar registrado y fallar 3 veces en el inicio de sesión.	
Dependencias:	CU-04. IniciarSesion	
Secuencia normal:	**N. º de paso**	**Acción**
	1	El usuario introduce tres veces mal sus datos de inicio de sesión
	2	La aplicación bloquea al usuario
	3	La aplicación envía un correo al usuario informando que su cuenta ha sido bloqueada y ofreciendo un enlace de un solo uso para restablecer su contraseña
	4	El usuario restablece su contraseña
Secuencia alternativa:	**N. º de paso**	**Acción**
		Ninguna
Postcondiciones:	Tras restablecer la contraseña, el usuario tiene que iniciar sesión con la nueva contraseña	
Importancia:	Media	
Comentario:	Únicamente se da cuando el usuario ha introducido tres veces mal el usuario, por lo tanto, debe de restablecer la contraseña y acceder con su nueva contraseña a través del enlace de un solo uso que se encuentra en el correo.	

ID/Nombre:	CU-07 Apuntarse_actividad
Actores:	Usuario
Objetivo:	El usuario puede apuntarse a una de las actividades previamente creadas en la aplicación.

Descripción:	Cuando el usuario busca una actividad y pulsa en 'apuntarse', la aplicación la apunta a esa actividad.	
Tipo:	Primario	
Precondiciones:	El usuario ha iniciado sesión en aplicación y ha buscado la actividad a la que quiere unirse.	
Dependencias:	CU-14 Crear Actividad	
Secuencia normal:	**N. º de paso**	**Acción**
	1	El usuario accede al buscador y busca la actividad a la que quiere apuntarse.
	2	La aplicación le muestra información detallada acerca de la actividad.
	3	El usuario pulsa sobre 'apuntarse' y la aplicación apunta a ese usuario en la actividad.
Secuencia alternativa:	**N. º de paso**	**Acción**
	3.1	El usuario no pulsa en 'apuntarse' y vuelve al listado de actividades para elegir otra
	3.2	La aplicación no le permite apuntarse a la actividad ya que no quedan plazas.
Postcondiciones:	El usuario se apunta a la actividad y la aplicación le envía un correo de confirmación	
Importancia:	Alta	
Comentario:	Cuando el usuario se apunta a una actividad, la aplicación envía un correo informativo acerca de la inscripción que acaba de realizar.	

ID/Nombre:	CU08. CrearEquipo
Actores:	UsuarioURJC
Objetivo:	Un usuario perteneciente a la URJC puede crear un equipo.

Descripción:	Un usuario vinculado a la URJC puede crear equipos para las actividades que lo requieran.	
Tipo:	Primario	
Precondiciones:	El usuario ha iniciado sesión en la aplicación.	
Dependencias:	CU-04 Iniciar_Sesión	
Secuencia normal:	**N. º de paso**	**Acción**
	1	El usuario inicia sesión en la aplicación y pulsa en 'crear equipo'
	2	La aplicación le muestra un formulario para rellenar los datos del equipo.
	3	El usuario completa el formulario y el equipo es creado.
Secuencia alternativa	**N. º de paso**	**Acción**
	3.1	Si el nombre del equipo ya existe pedirá al usuario que introduzca uno distinto.
Postcondiciones:	El usuario ha creado un equipo nuevo.	
Importancia:	Alta	
Comentario:	El usuario al crear equipo se convierte en el capitán del equipo, siendo el único miembro del equipo que puede modificar los datos y gestionar los integrantes del equipo. También existe la posibilidad de establecer un escudo del equipo.	

ID/Nombre:	CU-09 Unir equipo
Actores:	Usuarios
Objetivo:	Apuntarse a un equipo ya creado
Descripción:	Los usuarios registrados podrán apuntarse a un equipo previamente creado.
Tipo:	Primario

Precondiciones:	El usuario ha iniciado sesión y no pertenece a otro equipo.	
Dependencias:	CU-04 Iniciar_Sesión CU08. Crear_Equipo	
Secuencia normal	**N. º de paso**	**Acción**
	1	La aplicación muestra una lista con los equipos disponibles
	2	El usuario selecciona un equipo de la lista.
	3	La aplicación apunta al usuario en el equipo seleccionado
Secuencia alternativa	**N. º de paso**	**Acción**
	2.1	En caso de que el equipo este lleno, la aplicación le pedirá al usuario que vuelva a seleccionar un equipo diferente.
Postcondiciones:	El usuario queda apuntado a un equipo.	
Importancia:	Baja	
Comentario:	Los equipos solo serán necesarios para ciertas actividades cuando estas lo requieran. Los equipos tendrán un mínimo y máximo número de integrantes.	

ID/Nombre:	CU-10 Modificar equipo
Actores:	PAS, UsuarioURJC
Objetivo:	Modificar los datos de un equipo
Descripción:	Los usuarios PAS, así como los capitanes del equipo, podrán modificar los datos de un equipo (nombre, escudo, integrantes...)
Tipo:	Secundario

Precondiciones:	El usuario ha creado el equipo previamente	
Dependencias:	CU-08. Crear_Equipo	
Secuencia normal	**N. º de paso**	**Acción**
	1	El usuario inicia sesión y entra en la opción de modificar equipo.
	2	El usuario cambia los datos que desee o gestiona los participantes del equipo.
	3	El usuario acepta los cambios hechos.
Secuencia alternativa	**N. º de paso**	**Acción**
	2.1	El usuario cancela las modificaciones
Postcondiciones:	El equipo queda modificado	
Importancia:	Media	
Comentario:	En el caso de los UsuariosURJC, el usuario que modifique el equipo deberá ser el capitán. En caso de que el usuario que ha creado el equipo le abandone, el responsable será un usuario del equipo elegido aleatoriamente	

ID/Nombre:	CU-11 ConfirmarParticipación
Actores:	Usuario
Objetivo:	Confirmar la asistencia del usuario a la actividad
Descripción:	Un usuario que se apuntó a una actividad recibirá un correo el día de antes del inicio de dicha actividad.
Tipo:	Secundario
Precondiciones:	El usuario haya iniciado sesión y se haya apuntado a una actividad

Dependencias:	CU-04 Iniciar_Sesión CU-07 Apuntarse_Actividad	
Secuencia normal	**N. º de paso**	**Acción**
	1	El usuario inició sesión y se apuntó a una de las actividades que ofrece la aplicación
	2	Un día antes del inicio de la actividad se le manda un correo de forma de recordatorio
Secuencia alternativa	**N. º de paso**	**Acción**
	--	--
Postcondiciones:		
Importancia:	Baja	
Comentario:		

ID/Nombre:	CU-12 Modificar_Perfil	
Actores:	Usuario	
Objetivo:	Modificar los datos del perfil	
Descripción:	Los usuarios podrán modificar sus datos	
Tipo:	Secundario	
Precondiciones:	El usuario haya iniciado sesión previamente	
Dependencias:	CU-04 Iniciar_Sesion	
Secuencia normal	**N. º de paso**	**Acción**
	1	El usuario entrará en su perfil Pulsará sobre la opción de modificar perfil

	2	Modificará los datos
	3	Pulsará un botón de “Aceptar” para guardar los cambios
Secuencia alternativa	**N. º de paso**	**Acción**
	3	Pulsará el botón de “Cancelar” para no guardar los cambios
Postcondiciones:	El usuario habrá modificado sus datos correctamente	
Importancia:	Media	
Comentario:	Los usuarios podrán modificar sus datos tantas veces como quieran y cuando ellos quieran.	

ID/Nombre:	CU-13 Alquilar_Instalación	
Actores:	Usuario que ha creado el equipo	
Objetivo:	Alquilar instalaciones	
Descripción:	Los usuarios vinculados a la URJC pueden buscar y alquilar instalaciones a través de la aplicación.	
Tipo:	Primario	
Precondiciones:	Ser usuario miembro de la URJC y haber buscado y seleccionado una instalación.	
Dependencias:	CU-04 Iniciar_Sesión	
Secuencia normal:	**N. º de paso**	**Acción**
	1	El usuario URJC accede al buscador y busca una instalación
	2	La aplicación muestra una lista de las instalaciones que se pueden reservar.
	3	El usuario URJC selecciona una instalación

	4	La aplicación muestra la información detallada de la instalación y el usuario decide reservarla.
Secuencia alternativa:	**N. º de paso**	**Acción**
	3.1	Si la instalación no está disponible no se podrá realizar la reserva
Postcondiciones:	Ninguna	
Importancia:	Baja-Media	
Comentario:	Ninguno	
ID/Nombre:	CU-14 Crear_Actividad	
Actores:	Usuario PAS	
Objetivo:	Crear una nueva actividad en la que se pueden apuntar los usuarios de la aplicación.	
Descripción:	Los usuarios PAS con permisos de administración podrán hacer algunas gestiones en la aplicación. Entre estas gestiones se encuentra el crear nuevas actividades.	
Tipo:	Primario	
Precondiciones:	Ser usuario PAS con permisos de administración	
Dependencias:	CU-04 Iniciar_Sesion	
Secuencia normal:	**N. º de paso**	**Acción**
	1	El usuario PAS accede a la aplicación
	2	El usuario PAS accede a una opción única de los usuarios con permisos que le permite crear nuevas actividades
	3	El PAS introduce los datos de la actividad
	4	Se crea una nueva actividad que aparece al buscarla por su nombre
Secuencia alternativa:	**N. º de paso**	**Acción**

	3.1	Si se introduce un nombre de actividad que ya se encuentra registrado, la aplicación informará al PAS y no creará esa actividad.
Postcondiciones:	El usuario PAS crea una nueva actividad que se muestra al resto de usuarios en la lista de actividades que pueden elegir.	
Importancia:	Media	
Comentario:	Una descripción, un número de plazas, una fecha y una imagen son los datos que el PAS debe rellenar para cada actividad que desea crear.	

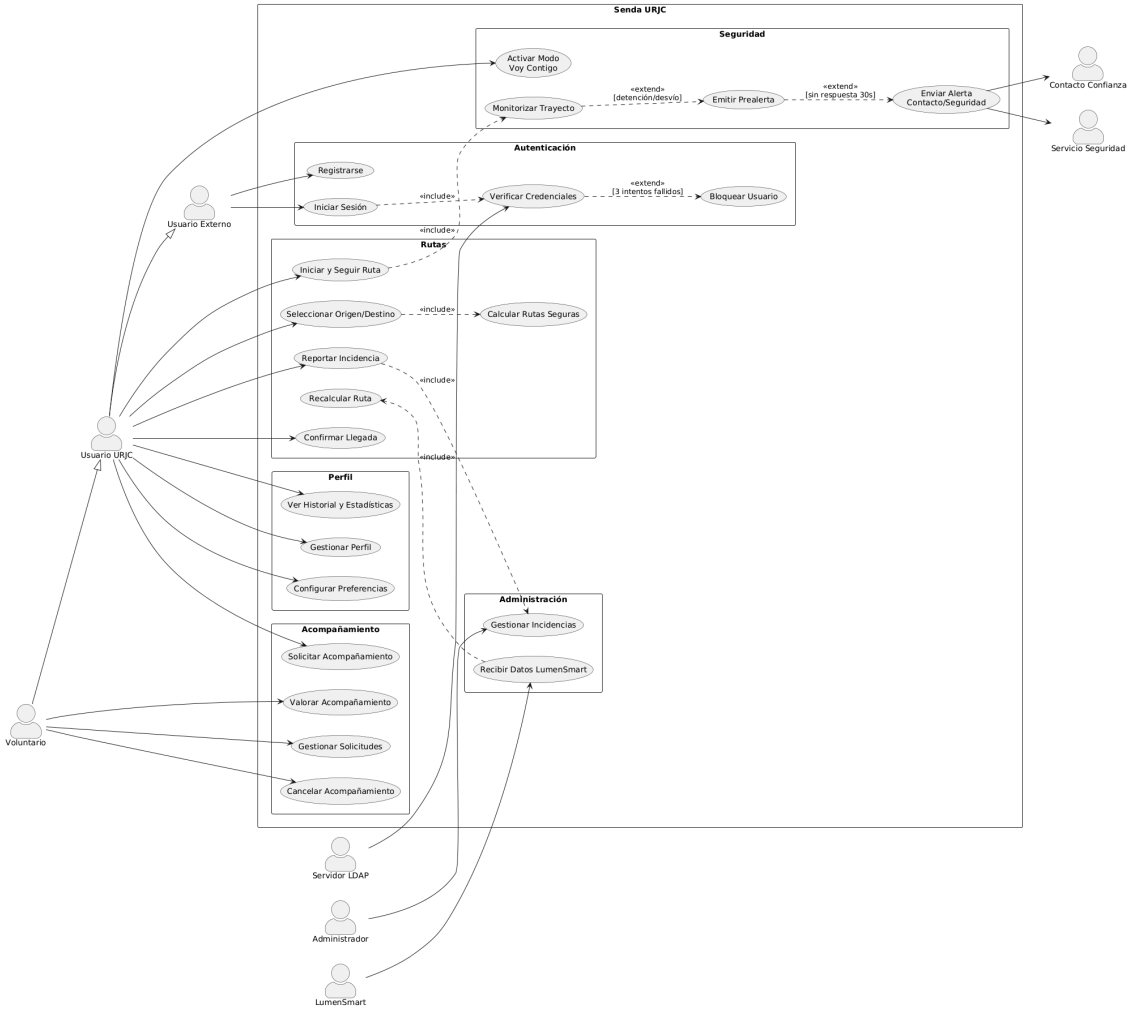
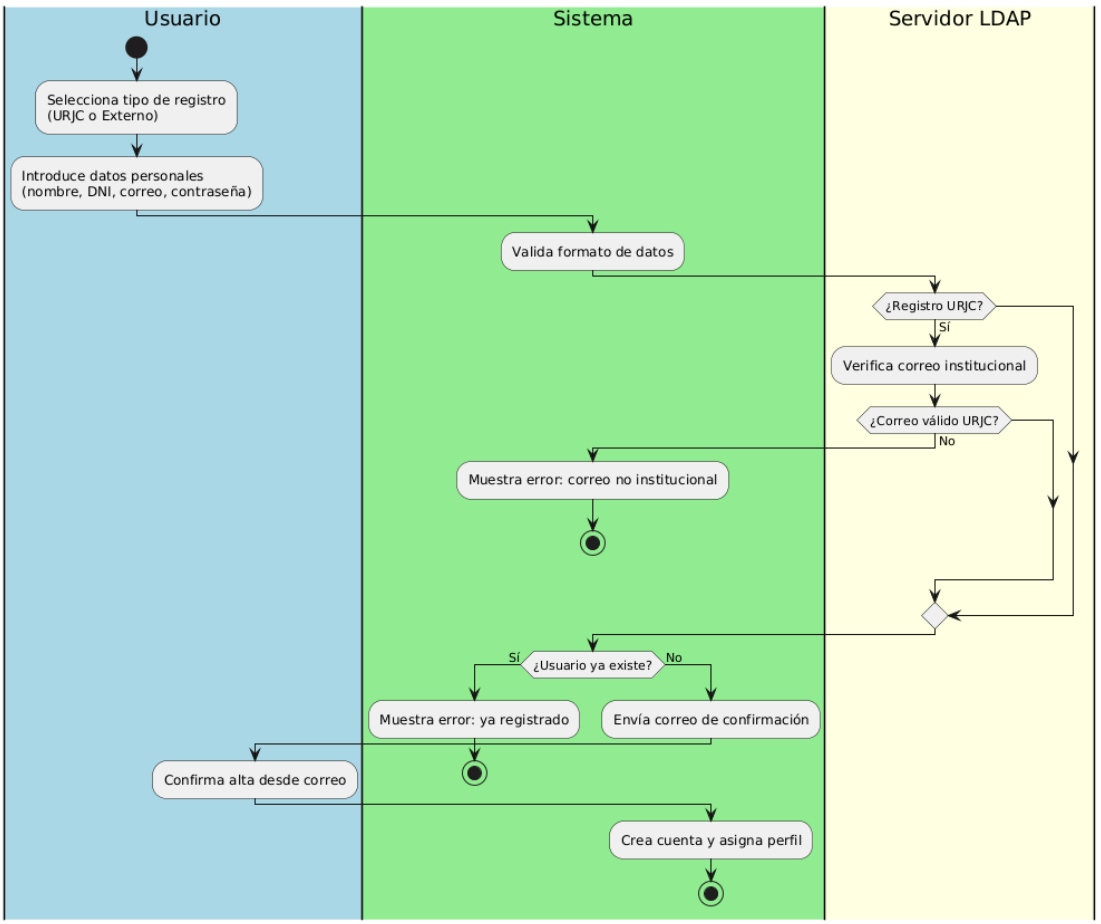
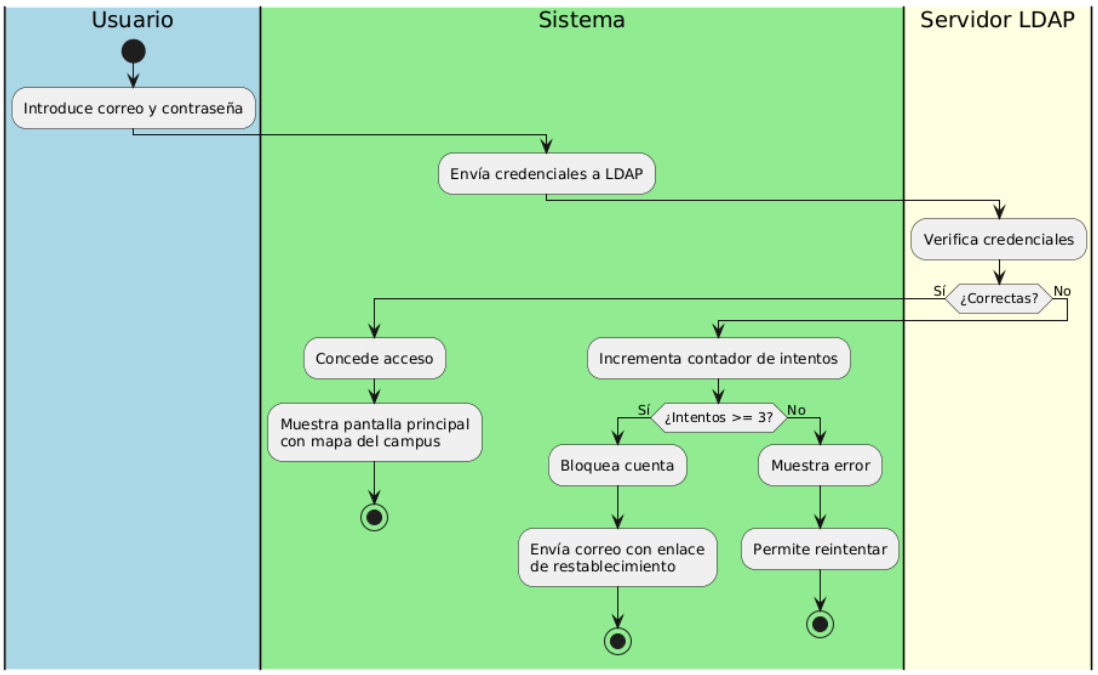


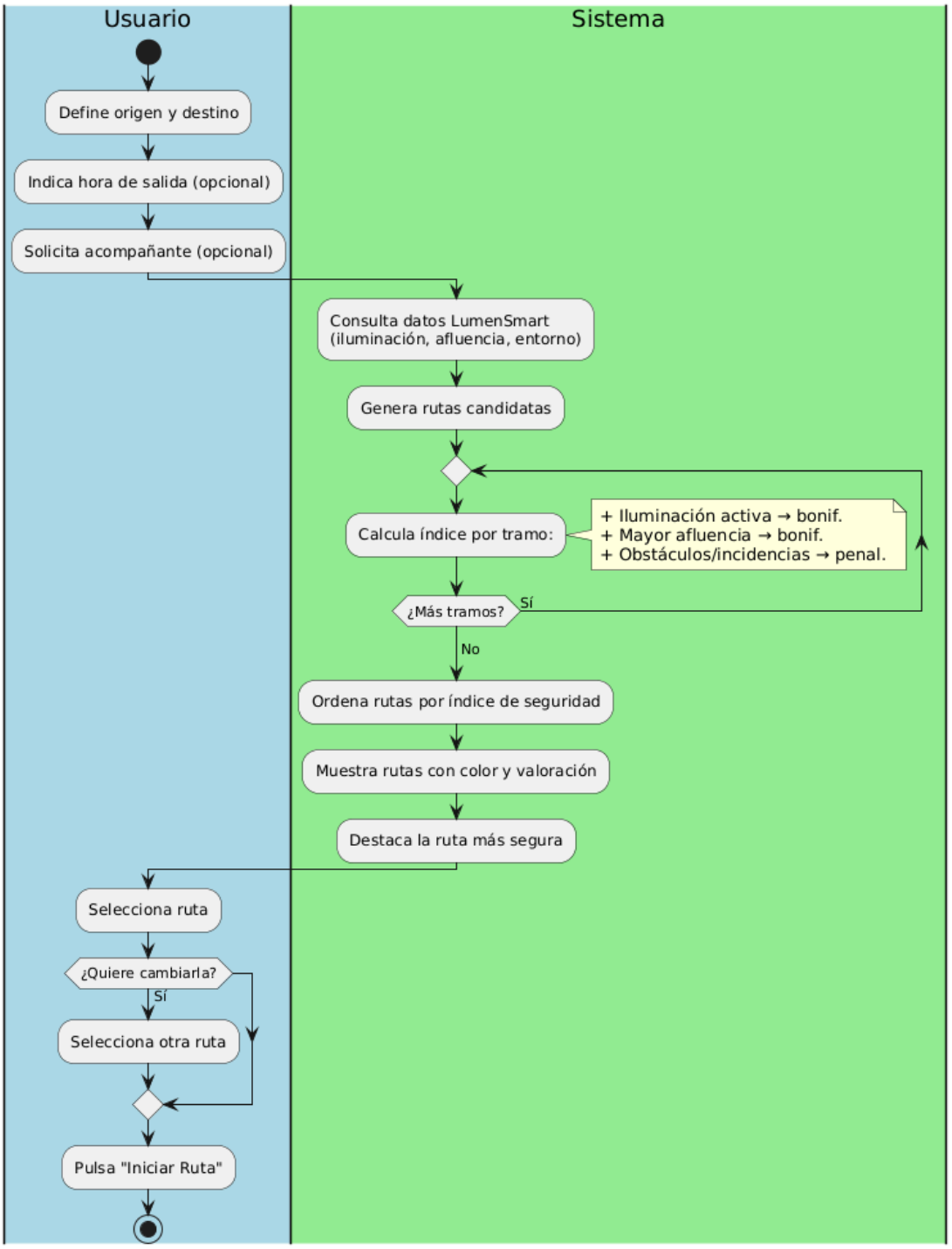
Diagrama de casos de uso



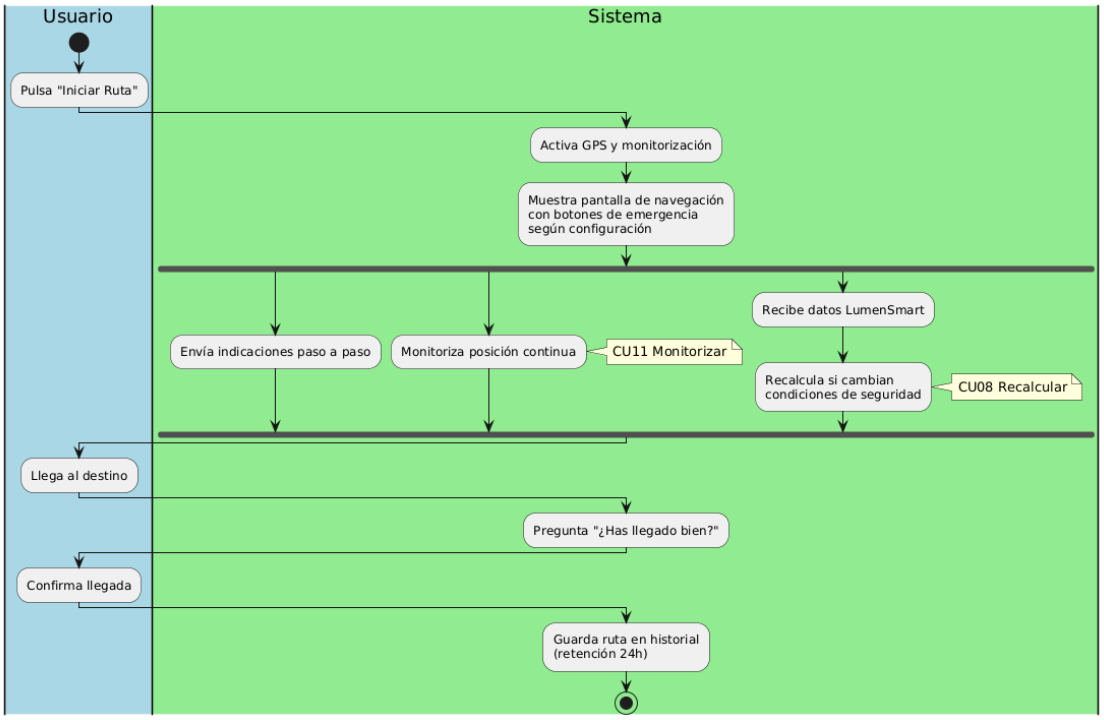
CU01 - Registrarse



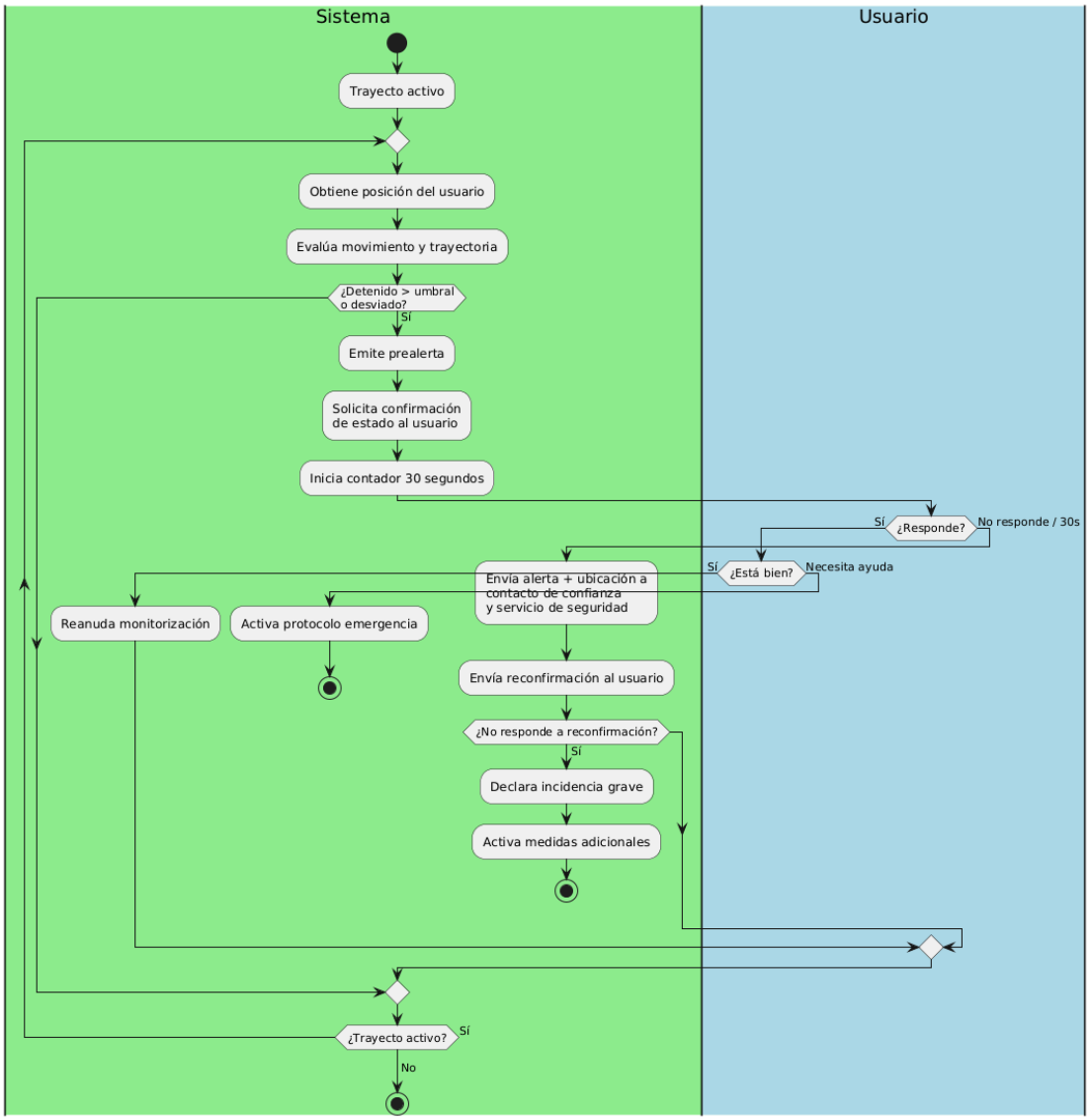
CU02 - Iniciar Sesión



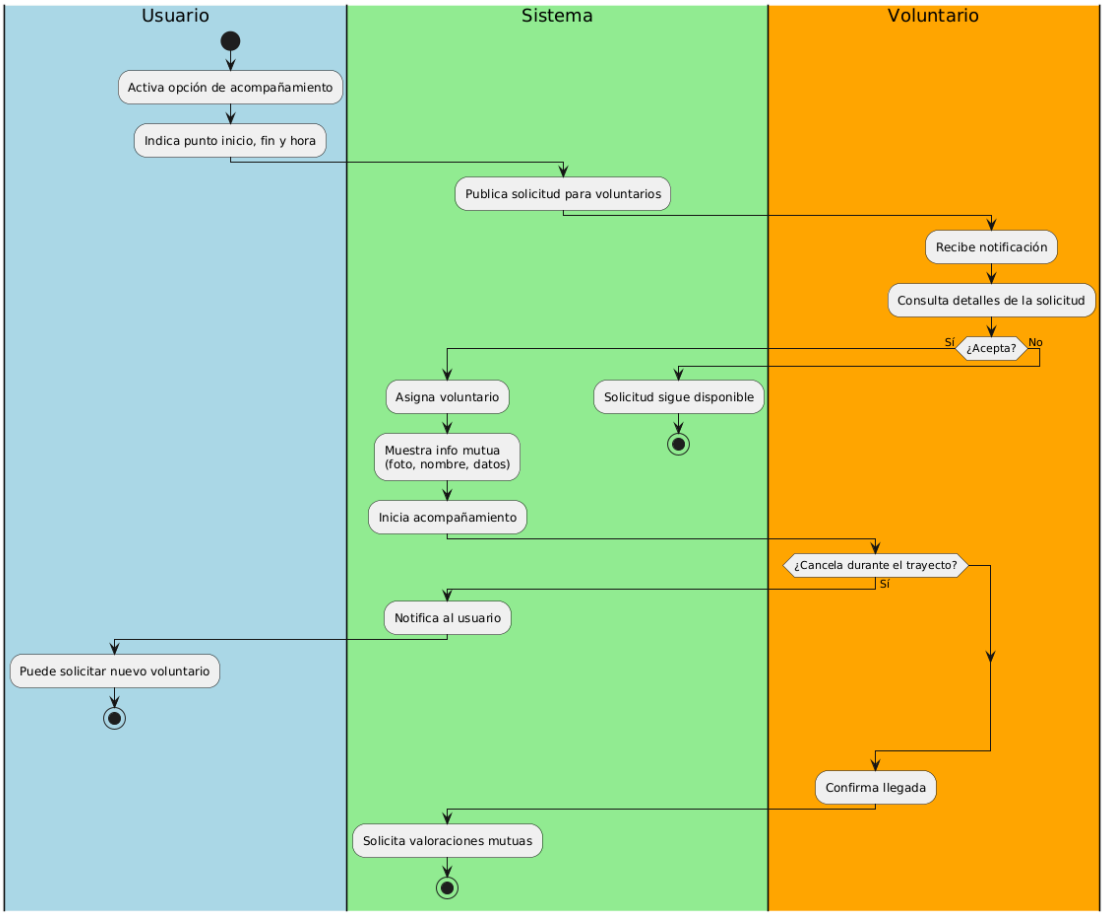
CU06 - Calcular Rutas Seguras



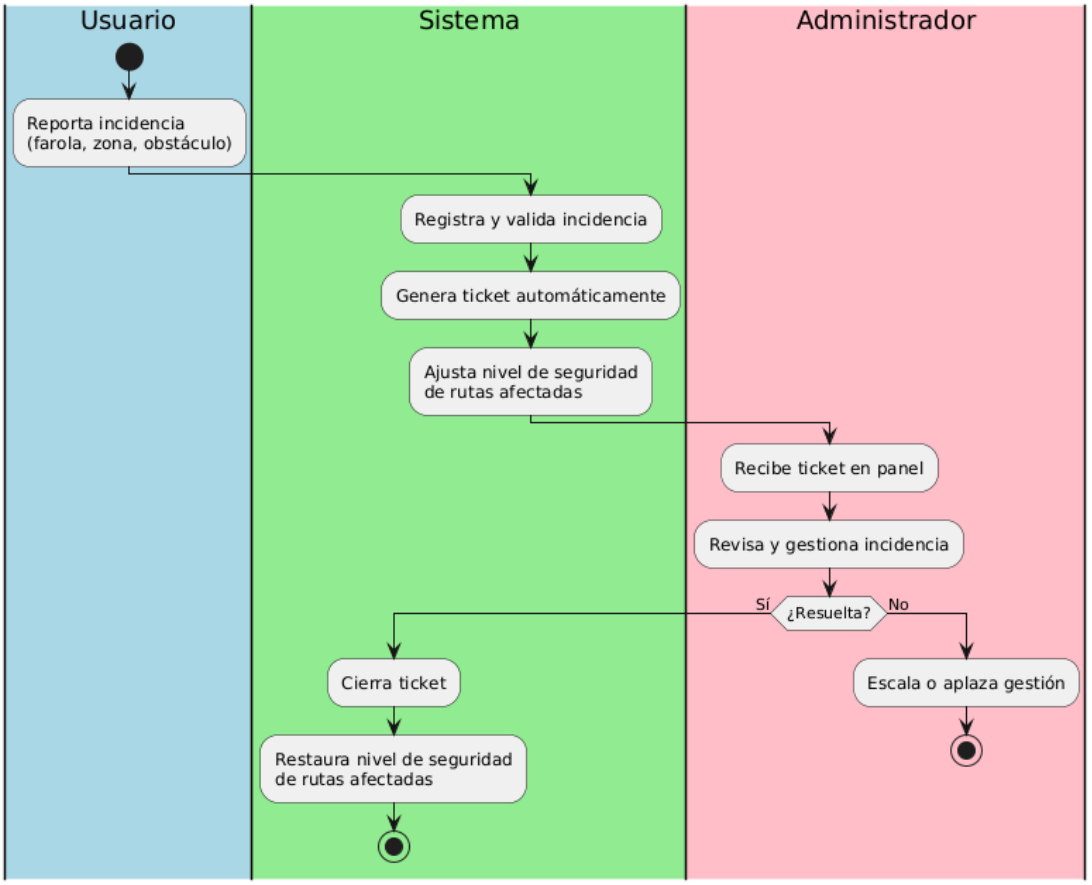
CU07 - Iniciar y Seguir Ruta



CU11/CU12/CU13 - Monitorizar, Prealerta y Alerta



CU15/CU16 - Solicitar y Gestionar Acompañamiento



CU22 - Gestionar Incidencias (Admin)

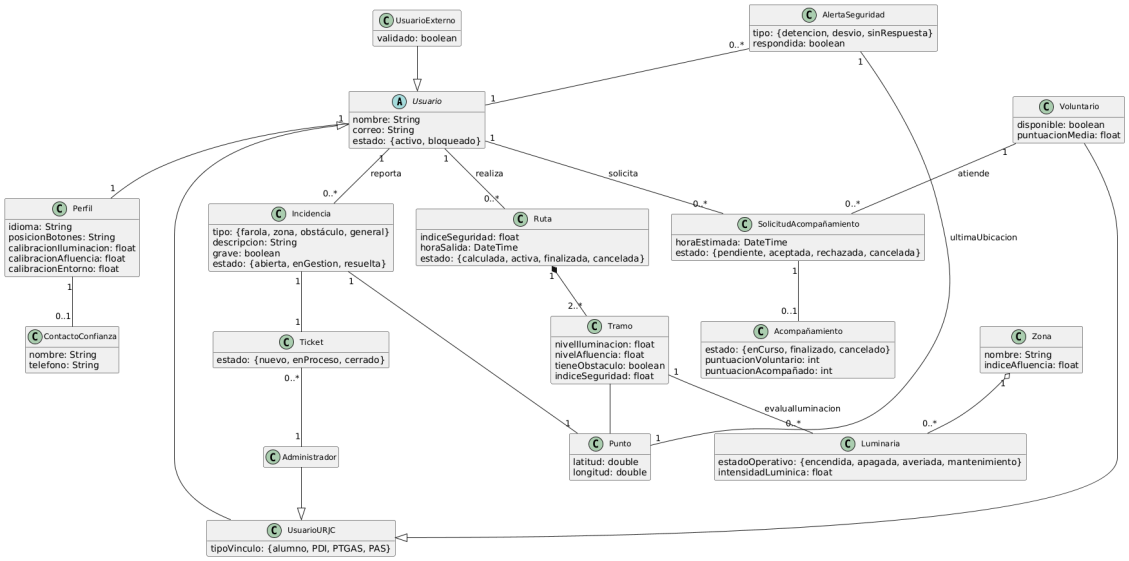
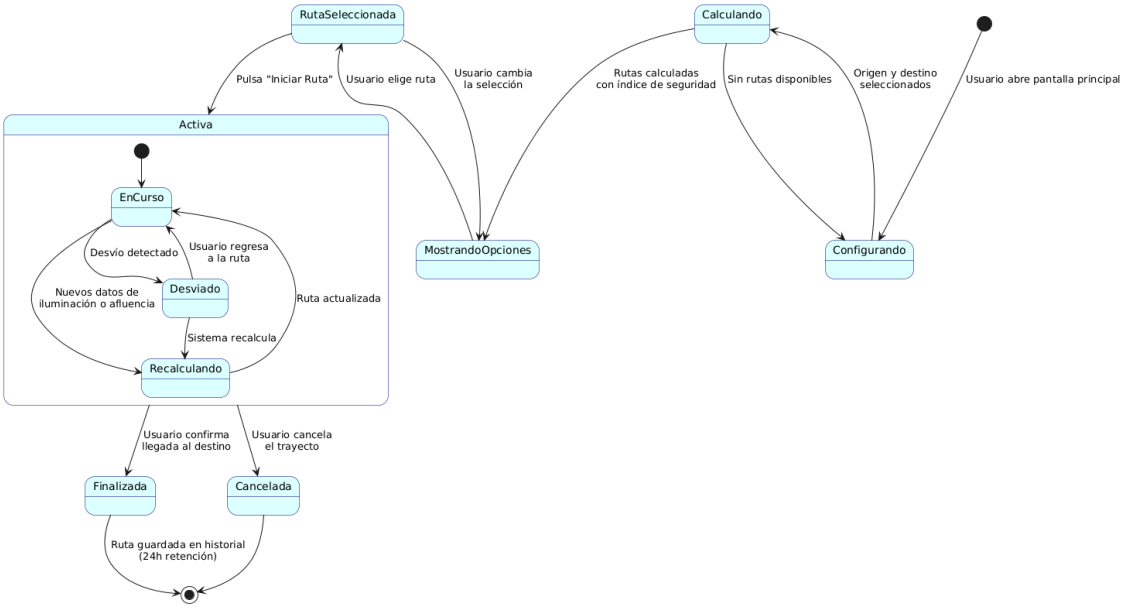
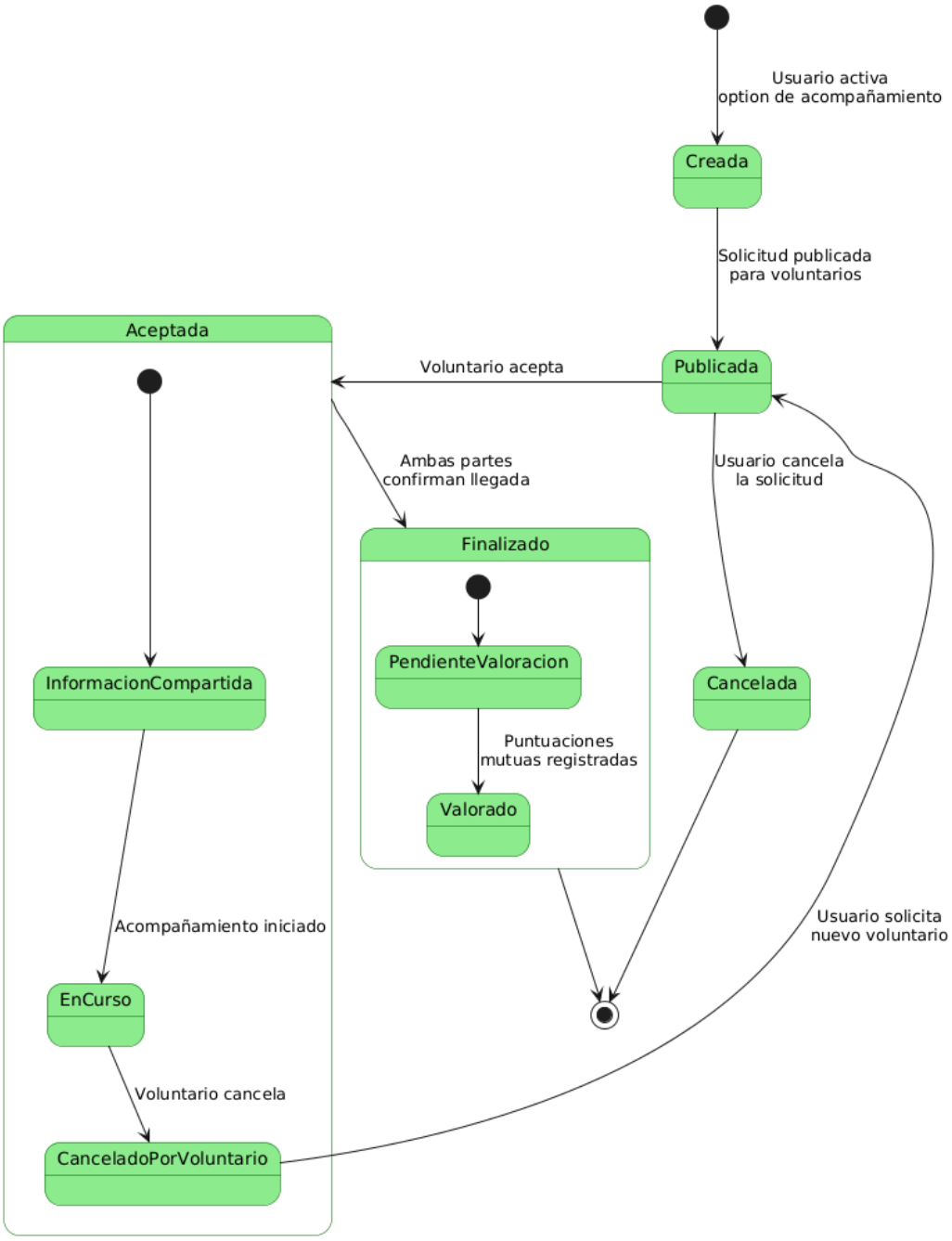


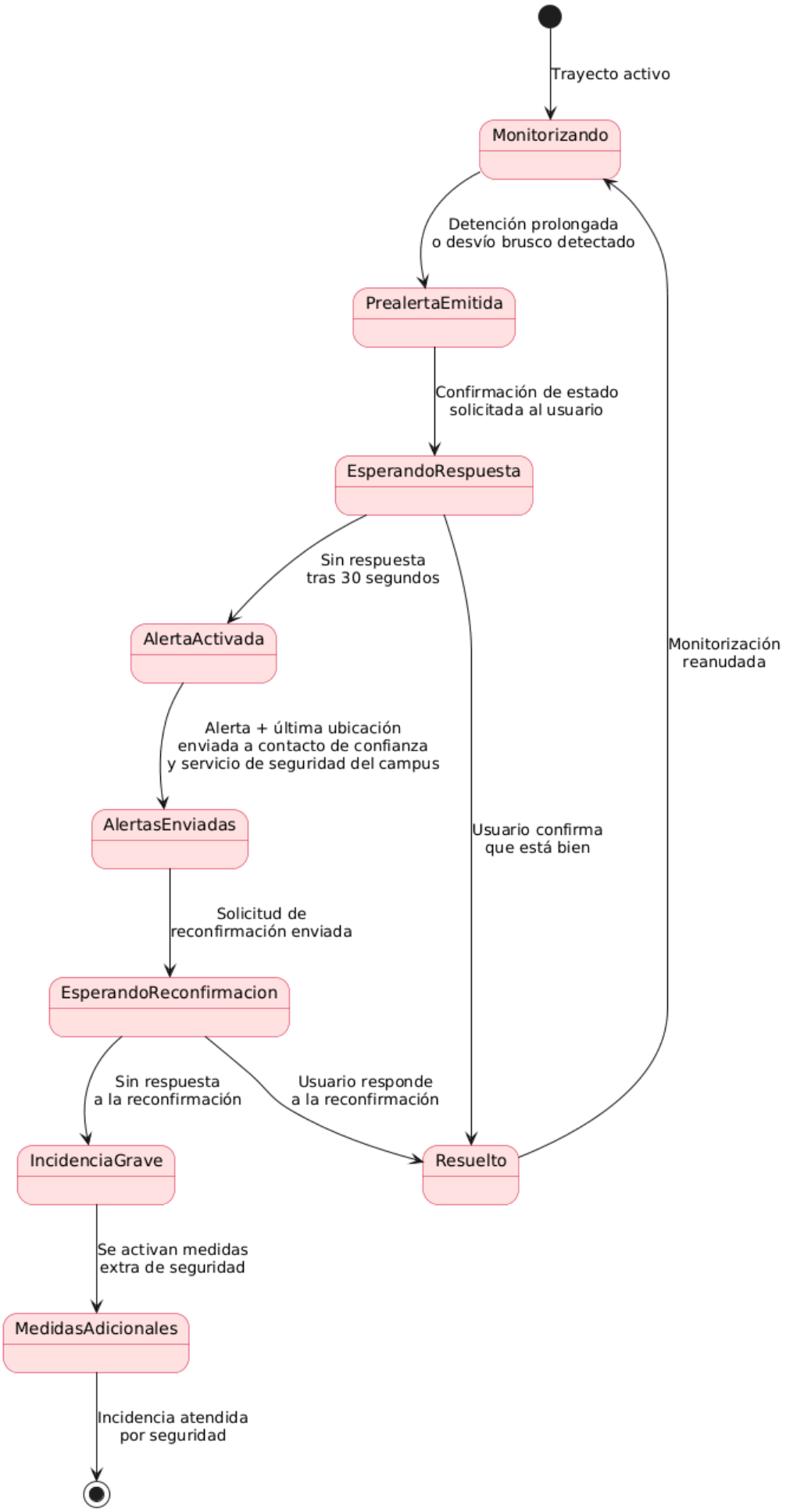
Diagrama de clases



Estado de la Ruta



Estado de la Solicitud de Acompañamiento


Trayecto activo
Monitorizando
Detención prolongada
o desvío brusco detectado
PrealertaEmitida
Confirmación de estado
solicitada al usuario
EsperandoRespuesta
Sin respuesta
tras 30 segundos
AlertaActivada
Alerta + última ubicación
enviada a contacto de confianza
y servicio de seguridad del campus
AlertasEnviadas
Solicitud de
reconfirmación enviada
EsperandoReconfirmacion
Sin respuesta
a la reconfirmación
Usuario responde
a la reconfirmación
IncidenciaGrave
Resuelto
Usuario confirma
que está bien
Monitorización
reanudada
Se activan medidas
extra de seguridad
MedidasAdicionales
Incidencia atendida
por seguridad